

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)
Факультет информационных технологий и кибербезопасности
Кафедра информационных систем и программирования

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению: 09.03.04 Программная инженерия

(код и наименование направления)

профиль: Проектирование и разработка программного обеспечения

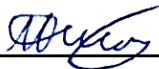
(наименование профиля)

на тему: «Разработка механизма гарантированной доставки сообщений с использованием паттерна Transactional Outbox в банковской сфере»

(наименование темы)

ФИТК.КИСП.09.03.04.020.ПП

(обозначение документа)

Автор  08.06.2026 / С.Н. Милькин /

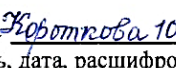
(подпись, дата, расшифровка подписи)

Руководитель  09.06.2026 / канд. экон. наук, доц. К.А. Ковалева /

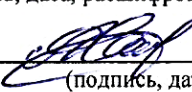
(подпись, дата, расшифровка подписи)

Консультанты:

Раздел безопасности и

экологичности проекта  10.06.26 / д-р техн. наук, проф. Т.Г. Короткова /

(подпись, дата, расшифровка подписи)

Нормоконтролер  11.06.2026 / ст. преп. А.А. Ковтун /

(подпись, дата, расшифровка подписи)

Выпускная квалификационная работа
допущена к защите

22.06.2026
(дата)

Заведующая кафедрой  / канд. техн. наук, доц. М.В. Янаева /

(подпись)

Краснодар
2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)
Факультет информационных технологий и кибербезопасности
Кафедра информационных систем и программирования

УТВЕРЖДАЮ

Заведующая кафедрой ИСП

к.т.н, доц. С.В. М.В. Янаева

«20» апреля 2026 г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

по направлению 09.03.04 Программная инженерия
(шифр и наименование)

профиль: Проектирование и разработка программного обеспечения
(наименование профиля)

студенту Милькину Святославу Николаевичу
(фамилия, имя, отчество)

Тема выпускной квалификационной работы: Разработка механизма
гарантированной доставки сообщений с использованием паттерна
Transactional Outbox в банковской сфере

Утверждена приказом ректора университета от 31.03.2026 г. № 684-Ст

Руководитель канд. экон. наук, доц. К. А. Ковалева
(должность, фамилия, инициалы)

Консультанты:

Раздел безопасности и
экологичности проекта д-р техн. наук, проф. Т.Г. Короткова
(должность, фамилия, инициалы)

Нормоконтролер ст. преп. А.А. Ковтун
(должность, фамилия, инициалы)

Срок сдачи выпускной квалификационной работы на кафедру 23.06.2026 г.

Содержание выпускной квалификационной работы

Введение

1 Нормативные ссылки

2 Термины и определения

3 Сокращения

4 Анализ предметной области

5 Разработка технического задания ПО

6 Проектирование структуры базы данных

7 Реализация базы данных средствами выбранной СУБД

8 Проектирование структуры приложения

9 Описание реализации программного продукта

10 Тестирование программного продукта и отладка

11 Проектирование технической информации к ПО

12 Безопасность и экологичность проекта

Заключение

Список использованных источников

Приложение А

Общее количество листов ПЗ 123

Объем иллюстративной части

1. Диаграммы UML.

2. ER-диаграммы.

3. Интерфейс ИС.

4. Видео работы проекта.

Общее количество слайдов иллюстративной части 11

Список основной и рекомендуемой литературы

1. Методические указания по выполнению выпускной квалификационной работы для студентов всех форм обучения и МИППС направлений 09.03.03 Прикладная информатика, 09.03.04 Программная

инженерия / Сост.: Л.А. Видовский, М.В. Янаева; Кубан. гос. технол. ун-т. Каф. информационных систем и программирования. – Краснодар: Изд. КубГТУ, 2016. – 44 с.

2. Лазарев, С. А. Проектирование высокоуровневой архитектуры распределенных информационных систем : учебное пособие / С. А. Лазарев, К. А. Польщиков. – Белгород : НИУ БелГУ, 2023. – 56 с.

3. Официальная документация Apache Kafka [Электронный ресурс]. – URL: <https://kafka.apache.org/documentation/>

Календарный план выполнения выпускной квалификационной работы

Месяц	Числа месяца																																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31		
Апрель																											Определение задач ВКР						
Май	Исследование предметной области							Описание требований к ПО					Построение модели Проектирование БД										ПО. Реализация БД. Реализация ПО. Тестирование.										
Июнь	Безопасность экологичность проекта.							и	Оформление пояснительной записки.					Нормоконтроль, антиплагиат. Сдача ВКР на кафедру										Защита ВКР									

Студент С.Н. Милькин
(подпись, дата)

Руководитель К.А. Ковалева
(подпись, дата)

Реферат

Выпускная квалификационная работа содержит 123 страницы, 20 рисунков, 7 таблиц, 8 источников, 1 приложение.

KOTLIN 1.9.25, JAVA 21, SPRING BOOT 3.5, POSTGRESQL 16, APACHE KAFKA, REST API, LIQUIBASE, TESTCONTAINERS, DOCKER, UML, PLANTUML

Объект разработки – событийно-ориентированная банковская платформа для обработки финансовых транзакций между двумя микросервисами.

Цель разработки – спроектировать и реализовать архитектуру микросервисной банковской платформы на основе паттернов CQRS и Transactional Outbox, обеспечивающую надёжную обработку финансовых операций, гарантированную доставку событий через Apache Kafka и согласованность данных между сервисами записи и чтения.

Предмет разработки – проектирование архитектуры, модели данных, механизмов надёжной доставки событий и обеспечения идемпотентности в распределённой банковской системе.

Практическая значимость состоит в реализации готовой к эксплуатации платформы с полным покрытием автоматическими тестами, документированным API и справочными материалами для развёртывания и сопровождения.

					ФИТК.КИСП.09.03.04.020.ПП			
Изм.	Лист	№ докум.	Подпись	Дата	Пояснительная записка выпускной квалификационной работы	Лит.	Лист	Листов
Разраб.		Милькин С.Н.		08.06.26				
Провер.		Ковалева К.А.		09.06.26			6	123
Консульт.		Короткова Т.Г.		10.06.26		КубГТУ		
Н. Конт		Ковтун А.А.		11.06.26				
Утверд.		Янаева М.В.		22.06.26				

Содержание

Введение.....	10
1 Нормативные ссылки	12
2 Термины и определения	14
3 Сокращения.....	17
4 Анализ предметной области	19
4.1 Общее описание предметной области	19
4.2 Назначение, цели и задачи создания ПО	19
4.3 Требования к структуре ПО и базе данных.....	20
4.4 Описание входных документов и сообщений.....	20
4.5 Описание выходных документов и сообщений	21
4.6 Список ограничений	21
5 Разработка технического задания ПО.....	23
5.1 Общие сведения	23
5.2 Назначение и цели создания (развития) системы.....	23
5.3 Характеристика объектов автоматизации	23
5.4 Требования к системе	24
5.4.1 Требования к внешним интерфейсам	24
5.4.2 Функциональные требования	25
5.4.3 Нефункциональные требования	26
5.5 Состав и содержание работ по созданию системы.....	27
5.6 Порядок контроля и приемки системы.....	28
5.7 Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие.....	28
5.8 Требования к документированию	29
6 Проектирование структуры базы данных.....	30
6.1 Описание сущностей	30
6.2 Построение концептуальной схемы базы данных.....	32
7 Реализация базы данных средствами выбранной СУБД	34

7.1 Выбор СУБД.....	34
7.2 Проектирование таблиц.....	34
7.3 Построение ER-диаграммы.....	36
8 Проектирование структуры приложения.....	38
8.1 Построение структурной схемы ПО	38
8.2 Построение архитектуры информационной системы	39
8.3 Разработка функциональной модели	39
8.4 Диаграмма компонентов	41
8.5 Диаграммы последовательности	42
8.6 Диаграмма размещения	43
8.7 Диаграмма состояний	44
8.8 Диаграммы потоков данных	44
8.9 Диаграмма прецедентов	45
8.10 Диаграмма кооперации.....	47
8.11 Диаграмма классов.....	47
9 Описание алгоритмов обработки данных.....	50
9.1 Описание алгоритмов обработки данных	50
9.2 Блок-схемы алгоритмов обработки данных	51
10 Описание реализации программного продукта	55
10.1 Выбор среды программирования	55
10.2 Последовательность разработки прикладной программы.....	56
10.3 Схема меню приложения	56
10.4 Описание окон приложения.....	57
10.5 Функциональные возможности программного приложения	58
10.6 Функции и процедуры обработки данных	59
11 Тестирование программного продукта и отладка	60
12 Проектирование технической информации к ПО.....	63
12.1 Требования к окружению и подключение.....	63
12.2 Основные REST endpoint.....	64
12.3 Формат сообщений для Kafka.....	66

12.4 Реализация паттерна Transactional Outbox	67
12.5 Мониторинг	68
13 Безопасность и экологичность проекта	70
13.1 Значение и задачи безопасности жизнедеятельности	70
13.2 Анализ условий труда и мероприятия по защите от воздействия вредных производственных факторов.....	72
13.3 Обеспечение электробезопасности	73
12.4 Пожарная безопасность.....	74
12.5 Безопасность жизнедеятельности в чрезвычайных ситуациях	75
Заключение	76
Список использованных источников	78
Приложение А Листинг программы.....	79
А.1 Конфигурация и сборка.....	79
А.2 Точка входа и инфраструктура	82
А.3 REST API – контроллеры	84
А.4 Сервисный слой – бизнес-логика	86
А.5 Репозитории – доступ к БД.....	100
А.6 Kafka – доставка	101
А.7 Модели	103
А.8 DTO – объект передачи данных	106
А.9 Модульные тесты	107

Введение

Выбор темы выпускной квалификационной работы связан с тем, что современные банковские и финансовые системы переходят от монолитной архитектуры к распределённым решениям на основе микросервисов. Обработка платежей, переводов и ведение истории операций – это область, в которой ошибки обходятся особенно дорого: потеря транзакции или её двойное выполнение недопустимы. Изучение того, как подобные задачи решаются на практике в промышленных системах, является актуальной задачей при подготовке к выпускной квалификационной работе.

Ключевая проблема, рассматриваемая в работе, состоит в том, что при асинхронном взаимодействии сервисов запись бизнес-данных в базу и публикация события в брокер сообщений не могут быть выполнены атомарно стандартными средствами. Если сервис завершится аварийно между этими операциями, событие будет потеряно, либо, наоборот, опубликовано без сохранения данных. Для решения этой проблемы в рамках практики применён паттерн Transactional Outbox: событие записывается в ту же базу данных в рамках одной транзакции с бизнес-данными, а отдельный процесс гарантированно доставляет его в Apache Kafka.

Второй методологической основой работы стал паттерн CQRS – разделение операций записи и чтения на два независимых сервиса. Сервис core-transaction принимает команды и проводит финансовые операции, сервис core-ledger потребляет события из Kafka, ведёт реестр транзакций и формирует отчёты в форматах PDF, DOCX и HTML.

В качестве технологического стека выбраны: язык Kotlin поверх JVM, фреймворк Spring Boot для построения REST-сервисов, СУБД PostgreSQL для хранения данных, Apache Kafka как брокер сообщений, Liquibase для версионирования схемы БД, Testcontainers для интеграционного тестирования, Docker для контейнеризации компонентов.

Целью выпускной квалификационной работы является проектирование и реализация событийно-управляемой платформы обработки банковских транзакций на основе архитектуры CQRS с применением паттерна Transactional Outbox.

Для достижения цели были поставлены следующие задачи:

- описать предметную область и сформировать техническое задание на разработку системы;
- спроектировать базы данных обоих сервисов и реализовать схему миграций через Liquibase;
- реализовать сервис core-transaction с поддержкой различных типов банковских операций и механизмом Transactional Outbox;
- реализовать сервис core-ledger с идемпотентным потреблением Kafka-событий и генерацией отчётов;
- разработать UML-диаграммы, отражающие архитектуру и поведение системы;
- описать ключевые алгоритмы обработки данных и покрыть реализацию автоматизированными тестами.

1 Нормативные ссылки

В настоящей выпускной квалификационной работе использованы ссылки на следующие нормативные документы:

1. ГОСТ Р 7.0.5-2008 СИБИД. Библиографическая ссылка. Общие требования и правила составления.
2. ГОСТ Р 7.0.108-2022 СИБИД. Библиографические ссылки на электронные документы, размещенные в информационно-телекоммуникационных сетях. Общие требования к составлению и оформлению.
3. ГОСТ Р ИСО 9241-210-2016 Эргономика взаимодействия человек – система.
4. ГОСТ 7.82-2001 СИБИД. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления.
5. ГОСТ 19.101-77 Единая система программной документации. Общие положения.
6. ГОСТ 19.401-78 ЕСПД. Текст программы. Требования к содержанию и оформлению.
7. ГОСТ 19.103-77 ЕСПД. Обозначение программ и программных продуктов.
8. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки.
9. ГОСТ 19.104-78. Единая система программной документации. Основные надписи.
10. ГОСТ 19.504-79. Единая система программной документации. Руководство программиста. Требования к содержанию и оформлению
11. ГОСТ 12.0.003-2015 ССБТ. Опасные и вредные производственные факторы. Классификация.

12. ГОСТ 12.1.019-2017 ССБТ. Электробезопасность. Общие требования и номенклатура видов защиты.

13. СанПиН 1.2.3685-21 Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания.

2 Термины и определения

В настоящей выпускной квалификационной работе применяются следующие термины с соответствующими определениями:

1. Информационная система (ИС) – совокупность программных, технических и организационных средств, предназначенных для сбора, хранения, обработки и предоставления информации пользователям.

2. Бэкенд (backend) – серверная часть веб-приложения, реализующая бизнес-логику, взаимодействие с базой данных, интеграцию с брокером сообщений и предоставление программного интерфейса (API) для клиентской части.

3. Фронтенд (frontend) – клиентская часть веб-приложения, исполняемая в браузере пользователя и обеспечивающая визуальный интерфейс взаимодействия с системой.

4. REST API (Representational State Transfer Application Programming Interface) – архитектурный стиль программного интерфейса, основанный на протоколе HTTP и предусматривающий обмен данными в формате JSON посредством стандартных методов: GET, POST, PATCH, DELETE.

5. Аутентификация – процедура проверки подлинности пользователя посредством предъявления учётных данных (электронной почты и пароля) с последующей идентификацией в запросах к API.

6. Асинхронная доставка – способ передачи сообщений, при котором операция отправки не блокирует ответ пользователю, а фактическая доставка выполняется в фоновом режиме через брокер сообщений.

7. Брокер сообщений – программный компонент промежуточного слоя, обеспечивающий приём, хранение и доставку сообщений между производителями (producer) и потребителями (consumer) в распределённой системе.

8. Apache Kafka – распределённая платформа потоковой передачи событий, используемая в разработанной системе в качестве брокера сообщений для асинхронной доставки уведомлений.

9. Топик (topic) – именованный канал в Kafka, в который публикуются сообщения определённого типа; в разработанной системе используется топик message-delivery для задач доставки писем.

10. Producer (производитель) – компонент системы, публикующий сообщения в топик Kafka при отправке письма или при повторной публикации задачи из outbox.

11. Consumer (потребитель) – компонент системы, подписанный на топик Kafka и обрабатывающий входящие сообщения доставки, создавая письмо во «Входящих» получателя.

12. Transactional Outbox (транзакционный outbox) – паттерн надёжной доставки, при котором запись о задаче доставки сохраняется в базе данных в той же транзакции, что и изменение состояния письма, с последующей публикацией в брокер сообщений.

13. Outbox (исходящая очередь, outbox-таблица) – в архитектуре информационных систем таблица или хранилище, в которое в рамках одной транзакции с основной бизнес-операцией записывается задача на последующую передачу данных во внешнюю систему (брокер сообщений, почтовый сервис, API), с тем чтобы доставка выполнялась асинхронно и не терялась при сбоях промежуточной инфраструктуры.

14. Идемпотентность доставки – свойство обработки сообщений, при котором многократное выполнение одной и той же операции доставки (в том числе при повторной обработке из-за сбоя или повторной публикации) даёт тот же результат, что и однократное выполнение, и не приводит к дублированию данных или побочным эффектам у получателя.

15. Слоистая архитектура – организация серверного приложения в виде уровней: REST API (контроллеры), сервисный слой (бизнес-логика), слой репозитория (доступ к данным) и модуль интеграции с Kafka.

16. Репозиторий – компонент слоя доступа к данным, инкапсулирующий операции чтения и записи в реляционную базу данных посредством JDBC.

17. UUID (Universally Unique Identifier) – стандартизованный 128-битный идентификатор, используемый в качестве первичных ключей сущностей в базе данных разработанной системы.

3 Сокращения

В данной выпускной квалификационной работе использованы следующие сокращения:

1. **ИС** – информационная система.
2. **ПО** – программное обеспечение.
3. **БД** – база данных.
4. **СУБД** – система управления базами данных.
5. **API (Application Programming Interface)** – программный интерфейс приложения.
6. **REST (Representational State Transfer)** – архитектурный стиль построения распределённых гипермедиасистем.
7. **HTTP (HyperText Transfer Protocol)** – протокол передачи гипертекста.
8. **JSON (JavaScript Object Notation)** – текстовый формат представления структурированных данных.
9. **JDBC (Java Database Connectivity)** – стандартный интерфейс доступа к реляционным базам данных из приложений на языке Java.
10. **UUID (Universally Unique Identifier)** – универсальный уникальный идентификатор.
11. **CRUD (Create, Read, Update, Delete)** – базовые операции с данными: создание, чтение, обновление, удаление.
12. **SQL (Structured Query Language)** – структурированный язык запросов.
13. **UML (Unified Modeling Language)** – унифицированный язык моделирования.
14. **DTO (Data Transfer Object)** – объект передачи данных между слоями приложения.
15. **BCrypt** – алгоритм хеширования паролей, применяемый при регистрации и аутентификации пользователей.

- 16. IDE (Integrated Development Environment)** – интегрированная среда разработки.
- 17. ВКР** – выпускная квалификационная работа.
- 18. ПК** – персональный компьютер.

4 Анализ предметной области

4.1 Общее описание предметной области

Предметная область проекта – обработка банковских финансовых операций. Пользователь системы – клиент банка, у которого есть один или несколько счетов: расчётный, сберегательный, депозитный или брокерский. К счёту может быть привязана платёжная карта, используемая при межбанковских переводах. Основное действие в системе – транзакция: событие, которое изменяет баланс одного или двух счетов.

Система поддерживает несколько типов операций: внутренние переводы между счетами, переводы по номеру телефона через СБП, межбанковские переводы по номеру карты, зачисление средств без списания – денежный подарок. Помимо выполнения операций, система ведёт полную историю транзакций и позволяет выгружать её в виде отчёта.

Ключевая сложность предметной области состоит в том, что банковская операция должна быть зафиксирована ровно один раз – без потерь и без дублирования. Система устроена так, что запись транзакции в базу данных и уведомление об этом смежного сервиса происходят атомарно: либо оба действия выполняются вместе, либо не выполняется ни одно из них.

4.2 Назначение, цели и задачи создания ПО

Назначение системы – надёжная обработка банковских транзакций с разделением операций записи и чтения на два независимых сервиса.

Цель – разработать и оформить документацию к платформе, которая гарантирует согласованность данных при асинхронном взаимодействии сервисов и корректно обрабатывает повторные или дублирующие запросы.

Для достижения цели поставлены следующие задачи:

- определить требования к системе и её интерфейсам;
- спроектировать базы данных обоих сервисов;

- разработать архитектуру приложения и подготовить UML-диаграммы;
- описать алгоритмы обработки транзакций и доставки событий между сервисами;
- зафиксировать поведение системы при недоступности брокера сообщений и при повторной доставке событий.

4.3 Требования к структуре ПО и базе данных

Система состоит из двух микросервисов с чёткими зонами ответственности, каждый из которых имеет собственную базу данных.

Первый сервис отвечает за приём и выполнение команд: создание пользователей и счетов, проведение финансовых операций. После успешного выполнения операции сервис формирует событие и гарантированно доставляет его второму сервису через брокер сообщений. Если брокер в этот момент недоступен, событие не теряется – оно сохраняется в базе данных и будет отправлено при первой возможности.

Второй сервис отвечает исключительно за чтение: он получает события от первого сервиса, сохраняет их в собственную базу данных и предоставляет REST API для запроса истории транзакций и формирования отчётов. Прямая запись через API в этот сервис не предусмотрена.

4.4 Описание входных документов и сообщений

Входными воздействиями для первого сервиса являются HTTP-запросы к REST API. Пользователь отправляет команды: зарегистрировать клиента, открыть счёт, выполнить финансовую операцию. Каждый запрос на проведение транзакции содержит уникальный идентификатор операции, сумму, необходимые реквизиты (номера счетов или карт, номер телефона – в зависимости от типа операции) и необязательное описание.

Входными сообщениями для второго сервиса являются события из топика Kafka. Каждое сообщение содержит идентификатор события, тип

операции, реквизиты счетов, сумму, валюту, статус транзакции и служебные метаданные: идентификатор агрегата, ключ партиционирования и время создания события.

4.5 Описание выходных документов и сообщений

Ответами первого сервиса на REST-запросы являются JSON-объекты. При создании пользователя возвращаются его данные и дата регистрации. При открытии счёта – тип счёта, валюта, баланс и сгенерированный номер счёта. При проведении транзакции – тип операции, статус, реквизиты и временные метки.

Ответами второго сервиса являются записи реестра транзакций: ссылка на исходную транзакцию, тип события и операции, реквизиты счетов, сумма, статус, время создания и время получения. Дополнительно сервис формирует отчёты по истории транзакций за выбранный период в форматах PDF, DOCX или HTML.

В случае ошибки оба сервиса возвращают единый формат ответа: HTTP-статус, текст ошибки и временная метка.

4.6 Список ограничений

Основные ограничения системы продиктованы требованиями финансовой консистентности.

Баланс счёта не может стать отрицательным. При недостатке средств операция отклоняется до внесения каких-либо изменений.

Каждый запрос на проведение транзакции должен содержать уникальный идентификатор. Повторный запрос с тем же идентификатором отклоняется, а не выполняется дважды. Аналогичная защита реализована при получении событий из Kafka: повторно доставленное сообщение распознаётся и пропускается.

При переводе между двумя счетами порядок их блокировки всегда фиксирован. Это исключает взаимную блокировку при одновременных встречных переводах между одними и теми же счетами.

Второй сервис является строго read-only с точки зрения внешнего API. Все данные в нём появляются исключительно через обработку событий из Kafka.

5 Разработка технического задания ПО

5.1 Общие сведения

Техническое задание описывает функциональные и нефункциональные требования к событийно-управляемой платформе обработки банковских транзакций, реализованной в виде двух микросервисов: core-transaction и core-ledger. Система предоставляет REST API для выполнения финансовых операций, а также для запроса истории транзакций и формирования отчётов. Взаимодействие между сервисами осуществляется асинхронно через брокер сообщений Apache Kafka.

Документ предназначен для разработчиков, аналитиков и специалистов по тестированию, принимающих участие в создании и сопровождении системы.

5.2 Назначение и цели создания (развития) системы

Система предназначена для надёжной обработки банковских транзакций с разделением операций записи и чтения, гарантированной доставки событий между сервисами и идемпотентного ведения реестра транзакций.

Цель данной работы – собрать и оформить проектную документацию по заданной структуре. В рамках этой цели описана предметная область, сформировано техническое задание, спроектированы базы данных и UML-диаграммы, а также зафиксированы ключевые алгоритмы обработки транзакций и поведение системы при частичных сбоях инфраструктуры.

5.3 Характеристика объектов автоматизации

Объектами автоматизации в системе являются следующие сущности предметной области:

- клиент банка и его персональные данные;
- банковский счёт с типом, валютой и балансом;

- платёжная карта, привязанная к счёту;
- финансовая транзакция и её атрибуты: тип, статус, сумма, реквизиты сторон;
- очередь гарантированной доставки событий (Outbox) и факты публикации в брокер;
- запись реестра транзакций – денормализованная read-модель на стороне core-ledger.

Дополнительным объектом автоматизации является сценарий асинхронной доставки: отправка события считается завершённой только после подтверждения успешной записи в базу данных core-ledger, а не в момент публикации в Kafka.

5.4 Требования к системе

5.4.1 Требования к внешним интерфейсам

Взаимодействие с системой осуществляется через REST API по протоколу HTTP. Для каждого сервиса доступен Swagger UI (/swagger-ui.html), предоставляющий интерактивную документацию по всем эндпоинтам с описанием полей запросов и ответов. Каждый сервис работает с собственной базой данных PostgreSQL 16, а асинхронное взаимодействие между сервисами обеспечивает Kafka-кластер из трёх брокеров. Все компоненты запускаются в Docker-контейнерах, а метрики сервисов экспонируются в формате Prometheus через эндпоинт /actuator/prometheus. При этом core-transaction предоставляет REST API для операций записи — управления пользователями и счетами, а также проведения транзакций, тогда как core-ledger предоставляет REST API только для операций чтения — запроса истории транзакций с фильтрацией и выгрузки отчётов. Взаимодействие между сервисами осуществляется через Kafka-топик core.transactions.request.

5.4.2 Функциональные требования

Функциональные требования определяют перечень операций, которые система обязана выполнять. Они охватывают управление клиентами и счетами, проведение финансовых операций различных типов, гарантированную доставку событий между сервисами, а также формирование истории транзакций и отчётности.

Функциональные требования представлены в таблице 1.

Таблица 1 – Функциональные требования

Номер	Требование
ФТ-1	Система должна обеспечивать создание клиента банка с указанием имени, фамилии, номера телефона и электронной почты.
ФТ-2	Система должна обеспечивать открытие счёта выбранного типа (расчётный, сберегательный, депозитный, брокерский) с автоматической генерацией уникального номера счёта.
ФТ-3	Система должна обеспечивать выполнение внутреннего перевода между двумя счетами с одновременным списанием у отправителя и зачислением получателю.
ФТ-4	Система должна обеспечивать выполнение перевода по номеру телефона через СБП.
ФТ-5	Система должна обеспечивать выполнение межбанковского перевода по номерам платёжных карт.
ФТ-6	Система должна обеспечивать зачисление средств на счёт без списания у отправителя (денежный подарок, компенсация).
ФТ-7	Система должна обеспечивать погашение кредита: списание с расчётного счёта клиента и зачисление на счёт назначения.
ФТ-8	После выполнения любой финансовой операции система должна атомарно сохранять событие в очередь доставки и гарантировать его публикацию в брокер сообщений.

Продолжение таблицы 1

ФТ-9	Система должна предоставлять историю транзакций по счёту с фильтрацией по дате, типу операции и статусу, а также с поддержкой постраничной навигации.
ФТ-10	Система должна формировать отчёт по истории транзакций за выбранный период в одном из форматов: PDF, DOCX или HTML.

Представленные функциональные требования охватывают полный жизненный цикл банковской операции: от регистрации клиента и открытия счёта до проведения транзакции, гарантированной доставки события в реестр и формирования итоговой отчётности.

5.4.3 Нефункциональные требования

Нефункциональные требования определяют ограничения и качественные характеристики системы, обеспечивающие финансовую корректность операций, согласованность данных при асинхронном взаимодействии сервисов и наблюдаемость системы в производственной среде.

Нефункциональные требования представлены в таблице 2.

Таблица 2 – Нефункциональные требования

Номер	Требование
НФТ-1	Баланс счёта не может стать отрицательным. Операция, приводящая к отрицательному балансу, должна быть отклонена до внесения каких-либо изменений в базу данных.
НФТ-2	Каждый запрос на проведение транзакции должен содержать уникальный идентификатор. Повторный запрос с тем же идентификатором должен быть отклонён, а не выполнен повторно.
НФТ-3	Повторно доставленное Kafka-сообщение не должно создавать дублирующую запись в реестре. Сервис core-ledger должен распознавать дубликат и пропускать его.

Продолжение таблицы 2

НФТ-4	При одновременном проведении операций по одним и тем же счетам система должна исключать взаимную блокировку за счёт фиксированного порядка захвата блокировок.
НФТ-5	Kafka-сообщение должно подтверждаться только после успешной записи в базу данных core-ledger. При сбое запись должна быть повторена в соответствии с настроенной политикой повторных попыток.
НФТ-6	Все изменения схемы баз данных должны выполняться через версионированные миграции. Запуск сервиса с несоответствующей схемой должен завершаться с ошибкой.
НФТ-7	Каждый сервис должен экспонировать метрики производительности и бизнес-показатели (количество транзакций по типу и статусу, количество дублирующих событий) в формате, совместимом с Prometheus.

Представленные нефункциональные требования обеспечивают финансовую корректность операций, согласованность данных при асинхронном взаимодействии и наблюдаемость системы в производственной среде.

5.5 Состав и содержание работ по созданию системы

Состав работ включает:

- анализ предметной области и формирование требований к системе;
- проектирование баз данных обоих сервисов и построение ER-диаграмм;
- реализацию сервиса core-transaction: REST API, бизнес-логика операций, компонент гарантированной доставки событий;
- реализацию сервиса core-ledger: Kafka-потребитель с идемпотентной обработкой, REST API истории транзакций, генерация отчётов;
- разработку UML-диаграмм, описывающих архитектуру и поведение системы;
- написание автоматизированных тестов и отладку по результатам;

- оформление пояснительной записки и технической документации.

Перечисленные работы охватывают полный цикл разработки – от анализа требований до тестирования и документирования и обеспечивают системный подход к созданию платформы.

5.6 Порядок контроля и приемки системы

Приёмка выполняется по итогам проверки ключевых сценариев:

- проведение каждого из поддерживаемых типов транзакций и проверка корректности изменения балансов;
- повторная отправка запроса с тем же идентификатором и проверка, что транзакция не была выполнена дважды;
- проверка появления записи в реестре core-ledger после успешного проведения операции в core-transaction;
- повторная доставка Kafka-сообщения и проверка, что дублирующая запись в реестре не создаётся;
- запрос истории транзакций с фильтрацией и формирование отчёта в каждом из поддерживаемых форматов.

5.7 Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие

Подготовка к вводу в действие предполагает развёртывание следующих компонентов инфраструктуры:

- kafka-кластер из трёх брокеров с созданием топика core.transactions.request;
- база данных transaction_db для сервиса core-transaction с применением всех миграций;
- база данных ledger_db для сервиса core-ledger с применением всех миграций.

Все компоненты запускаются через Docker Compose. Сервисы не имеют общих баз данных и не обращаются напрямую к базам данных друг друга.

5.8 Требования к документированию

Документация включает:

- техническое задание с функциональными и нефункциональными требованиями;
- описание сущностей и схем баз данных обоих сервисов;
- архитектурные и поведенческие UML-диаграммы;
- описание алгоритмов обработки транзакций и доставки событий;
- техническую справку: перечень REST-эндпоинтов, формат Kafka-сообщений, инструкция по запуску.

6 Проектирование структуры базы данных

6.1 Описание сущностей

Проектирование структуры базы данных выполнено на основании требований предметной области. Система использует две независимые базы данных: transaction_db обслуживает сервис обработки транзакций, ledger_db – сервис ведения реестра. Основные сущности представлены в таблицах 3-5.

Таблица 3 – Назначение сущностей

Сущность	Назначение
users	Клиенты банка: идентификатор, имя, фамилия, отчество, номер телефона (уникальный), электронная почта, дата создания и обновления.
accounts	Банковские счета: идентификатор, ссылка на клиента, номер счёта (уникальный, 20 символов), тип счёта, валюта, баланс (NUMERIC(19,4), не менее 0), признак активности, даты создания и обновления. Связан с users по user_id.
cards	Платёжные карты: идентификатор, ссылка на счёт, номер карты (уникальный), дата истечения, признак активности, дата создания. Связана с accounts по account_id.
transactions	Финансовые транзакции: идентификатор, ключ идемпотентности (UUID, уникальный), тип операции, статус, ссылки на счёт-источник (nullable) и счёт-получатель, сумма, валюта, описание, текст ошибки, дата создания и завершения.
outbox_messages	Очередь событий для гарантированной доставки: идентификатор, тип и идентификатор агрегата, тип события, ключ партиционирования, номер партиции, полезная нагрузка в формате JSONB, дата создания. Составной первичный ключ (id, partition_num). Таблица секционирована по списку: 8 физических партиций (p0–p7).

Продолжение таблицы 3

outbox_consumer_offsets	Смещения потребителей Outbox: идентификатор, наименование группы потребителей, номер партии, последнее обработанное смещение, дата обновления. Уникальное ограничение на пару (consumer_group, partition_num).
outbox_partition_locks	Блокировки партий: идентификатор, группа потребителей, номер партии, идентификатор захватившего блокировку процесса, время захвата и время истечения. Уникальное ограничение на пару (consumer_group, partition_num).

Таблица 4 – Описание сущностей ledger_db

Сущность	Назначение
transaction_ledger	Денормализованная read-модель реестра транзакций: идентификатор, ключ идемпотентности (уникальный), тип события, идентификатор исходной транзакции, тип операции, номера счетов отправителя (nullable) и получателя, сумма, валюта, статус, идентификатор агрегата, ключ партиционирования, идентификатор Outbox-сообщения, время создания события и время его получения.

Таблица 5 – Описание связей transaction_db

Связь	Назначение
accounts → users (user_id)	Счёт принадлежит клиенту; у одного клиента может быть несколько счетов.
cards → accounts (account_id)	Карта привязана к счёту; у одного счёта может быть несколько карт.
transactions → accounts (source_account_id)	Транзакция ссылается на счёт-источник; поле nullable – для операций без списания (денежный подарок, компенсация).
transactions → accounts (destination_account_id)	Транзакция ссылается на счёт-получатель; обязательное поле для всех типов операций.

Сущности outbox_messages, outbox_consumer_offsets и outbox_partition_locks не имеют внешних ключей на бизнес-таблицы – это

намеренное архитектурное решение: Outbox-компонент работает независимо и не создаёт связей, которые могли бы заблокировать удаление бизнес-данных.

База данных ledger_db содержит единственную таблицу transaction_ledger без внешних ключей. Это денормализованная модель: все необходимые данные хранятся в одной строке, что исключает JOIN-запросы при чтении истории и ускоряет выборку.

6.2 Построение концептуальной схемы базы данных

Концептуальная схема построена для отображения связей между сущностями предметной области. Поскольку система использует две независимые базы данных, схема разделена на два блока. Связь между ними логическая: transaction_ledger.source_transaction_id ссылается на идентификатор транзакции из transaction_db, однако на уровне базы данных это ограничение не закреплено (рис. 1).

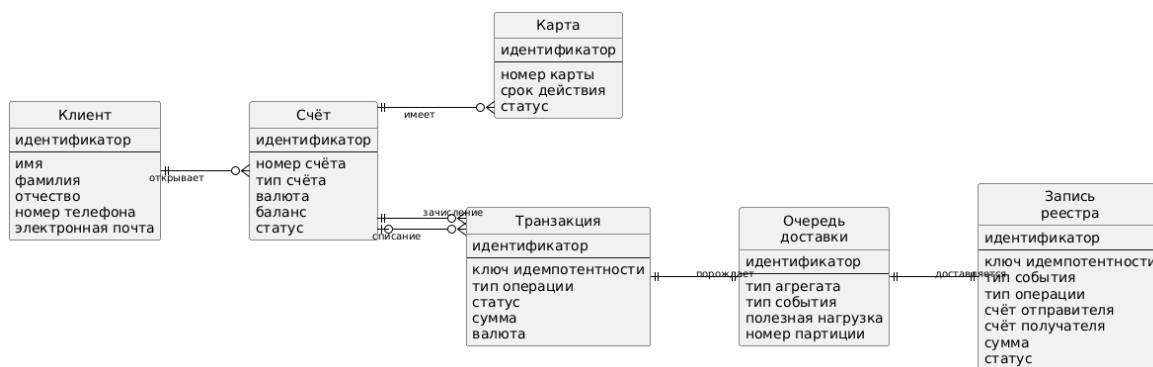


Рисунок 1 – Концептуальная схема

На концептуальной схеме отражены все сущности предметной области. В базе transaction_db центральной сущностью является transactions, связанная со счетами через два внешних ключа. Сущности outbox_messages, outbox_consumer_offsets и outbox_partition_locks обеспечивают работу механизма гарантированной доставки и намеренно не связаны с бизнес-таблицами внешними ключами. База ledger_db содержит единственную

денормализованную таблицу transaction_ledger, связанную с transaction_db логически через идентификатор исходной транзакции, передаваемый в Kafka-сообщении.

7 Реализация базы данных средствами выбранной СУБД

7.1 Выбор СУБД

Для хранения данных обоих сервисов выбрана СУБД PostgreSQL 16. Этот выбор обусловлен несколькими причинами. Во-первых, PostgreSQL обеспечивает надёжную транзакционную модель с полной поддержкой ACID, что критично для банковской предметной области: списание и зачисление должны фиксироваться атомарно. Во-вторых, PostgreSQL поддерживает тип данных JSONB, который используется для хранения полезной нагрузки Outbox-событий – это позволяет сохранять произвольную структуру события без дополнительных таблиц. В-третьих, PostgreSQL поддерживает LIST-секционирование таблиц, что применяется для разделения таблицы outbox_messages на 8 физических партиций для параллельной обработки. Наконец, PostgreSQL поддерживает пессимистичные блокировки (SELECT ... FOR UPDATE), которые используются для безопасного изменения балансов при конкурентных переводах.

Управление схемой осуществляется через инструмент миграций Liquibase: каждое изменение оформлено как SQL-файл, подключённый через XML-чейнджлог. Hibernate работает в режиме validate – приложение не создаёт и не изменяет таблицы самостоятельно, а лишь проверяет соответствие схемы при запуске.

7.2 Проектирование таблиц

Проектирование таблиц выполнено с учётом связей между сущностями предметной области. Система использует две изолированные базы данных.

База данных transaction_db (порт 5434) содержит семь таблиц и пять последовательностей:

- таблица users хранит данные клиентов банка: имя, фамилия, отчество, номер телефона (уникальный) и электронная почта. Первичный ключ – автоинкрементный BIGSERIAL;

– таблица accounts хранит банковские счета со связью с users по user_id. Номер счёта (account_number, 20 символов) генерируется автоматически через PostgreSQL-последовательности: seq_account_checking, seq_account_savings, seq_account_deposit, seq_account_brokerage – по одной на каждый тип. Баланс хранится как NUMERIC(19,4) с ограничением CHECK (balance >= 0). Допустимые типы счетов ограничены через CHECK (account_type IN ('CHECKING', 'SAVINGS', 'DEPOSIT', 'BROKERAGE'));

– таблица cards хранит платёжные карты со связью с accounts по account_id. Номер карты уникален, срок действия хранится как DATE. Номера карт генерируются через последовательность seq_card;

– таблица transactions хранит финансовые операции. Ключ идемпотентности (idempotency_key, UUID) имеет ограничение UNIQUE. Статус ограничен через CHECK (status IN ('PENDING', 'COMPLETED', 'FAILED', 'CANCELLED')). Тип операции ограничен восемью допустимыми значениями. Поле source_account_id допускает NULL – для операций без списания. Сумма проверяется ограничением CHECK (amount > 0);

– таблица outbox_messages реализует Transactional Outbox. Составной первичный ключ (id, partition_num). Таблица секционирована по списку (PARTITION BY LIST (partition_num)) на 8 физических партиций (outbox_messages_p0 – outbox_messages_p7). Полезная нагрузка хранится в поле payload типа JSONB;

– таблица outbox_consumer_offsets хранит последнее обработанное смещение для каждой пары (consumer_group, partition_num) с ограничением уникальности;

– таблица outbox_partition_locks реализует лизинговую блокировку партиций: каждая запись содержит идентификатор захватившего процесса, время захвата и время истечения. Уникальное ограничение на пару (consumer_group, partition_num).

База данных ledger_db (порт 5435) содержит одну таблицу transaction_ledger – денормализованная read-модель. Ключ идемпотентности

(idempotency_key, VARCHAR(255)) имеет ограничение UNIQUE для защиты от дубликатов. Таблица содержит все необходимые данные в одной строке: тип события, идентификатор исходной транзакции, тип операции, номера счетов, сумму, валюту, статус, метаданные агрегата и временные метки. Проиндексированы поля source_transaction_id, source_account_number, destination_account_number и event_created_at для быстрой выборки истории.

7.3 Построение ER-диаграммы

ER-диаграмма построена на основании реальных таблиц и их внешних ключей. Поскольку система использует две независимые базы данных, диаграммы представлены отдельно для каждой (рис. 2-3).

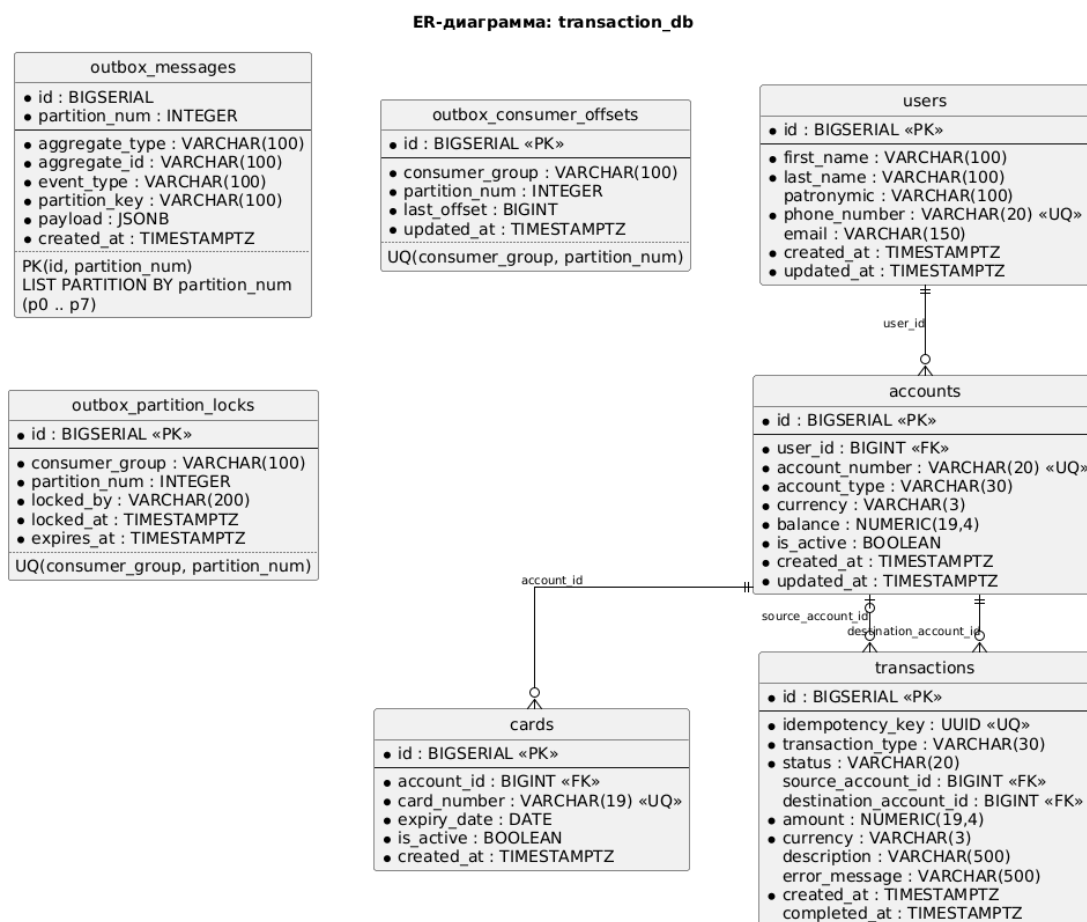


Рисунок 2 – ER-диаграмма transaction_db

ER-диаграмма: ledger_db

transaction_ledger
• id : BIGSERIAL «PK» • idempotency_key : VARCHAR(255) «UQ» • event_type : VARCHAR(50) • source_transaction_id : BIGINT «IDX» • transaction_type : VARCHAR(30) • source_account_number : VARCHAR(20) «IDX» • destination_account_number : VARCHAR(20) «IDX» • amount : NUMERIC(19,4) • currency : VARCHAR(3) • status : VARCHAR(20) • aggregate_id : VARCHAR(255) • partition_key : VARCHAR(255) • source_message_id : BIGINT • event_created_at : TIMESTAMPTZ «IDX» • received_at : TIMESTAMPTZ

Рисунок 3 – ER-диаграмма ledger_db

На ER-диаграмме transaction_db отображены все семь таблиц и связи между ними. Центральная бизнес-сущность – transactions – связана со счетами двумя внешними ключами: source_account_id (nullable для операций без списания) и destination_account_id. Счета связаны с клиентами, карты – со счетами. Таблицы Outbox (outbox_messages, outbox_consumer_offsets, outbox_partition_locks) намеренно не имеют внешних ключей на бизнес-таблицы: Outbox-компонент работает автономно, и отсутствие FK позволяет очищать старые сообщения без каскадных зависимостей.

8 Проектирование структуры приложения

8.1 Построение структурной схемы ПО

Диаграмма пакетов в UML показывает крупные блоки системы (пакеты, модули) и зависимости между ними. Она используется, чтобы увидеть декомпозицию проекта на уровне слоёв и подсистем, направление зависимостей и степень связанности между частями (рис. 4).

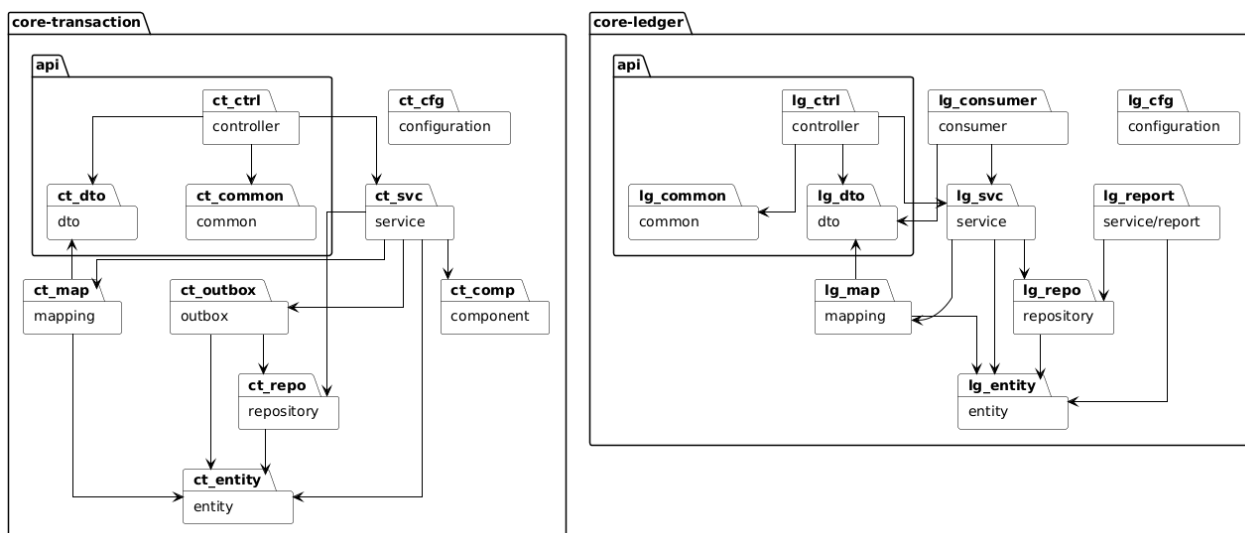


Рисунок 4 – Структурная схема ПО

На диаграмме отражена структура обоих сервисов. В core-transaction контроллеры принимают запросы и передают управление в слой сервисов, который содержит бизнес-логику и взаимодействует с репозиториями, маппингом и Outbox-компонентом для публикации событий. В core-ledger аналогичная слоистая структура дополнена пакетом consumer (Kafka-потребитель), который вызывает сервис для обработки событий, и пакетом service/report для генерации отчётов. Направление зависимостей строго сверху вниз: контроллеры зависят от сервисов, сервисы – от репозитория и entity, нижние слои не зависят от верхних.

8.2 Построение архитектуры информационной системы

Архитектура информационной системы основывается на принципе разделения ответственности (CQRS). Сервис core-transaction выполняет роль стороны записи: REST API принимает команды, сервисный слой реализует бизнес-логику транзакций, репозитории обеспечивают доступ к данным, а компонент Outbox гарантирует доставку событий в Kafka. Сервис core-ledger выполняет роль стороны чтения: Kafka-потребитель получает события и сохраняет их в денормализованную модель, а REST API предоставляет историю транзакций и генерирует отчёты.

Подробное отображение компонентов и зависимостей представлено далее в диаграмме компонентов (п. 8.4).

Общая архитектура обеспечивает сопровождаемость: изменения в правилах обработки транзакций затрагивают только сервисный слой core-transaction, а изменения в логике запросов истории – только core-ledger.

8.3 Разработка функциональной модели

Диаграмма деятельности в UML отображает последовательность действий, решения (ветвления), параллельные потоки и переходы. Выбран основной сценарий работы системы – проведение финансовой операции от момента запроса до появления записи в реестре (рис. 5).

Функциональная модель: проведение транзакции

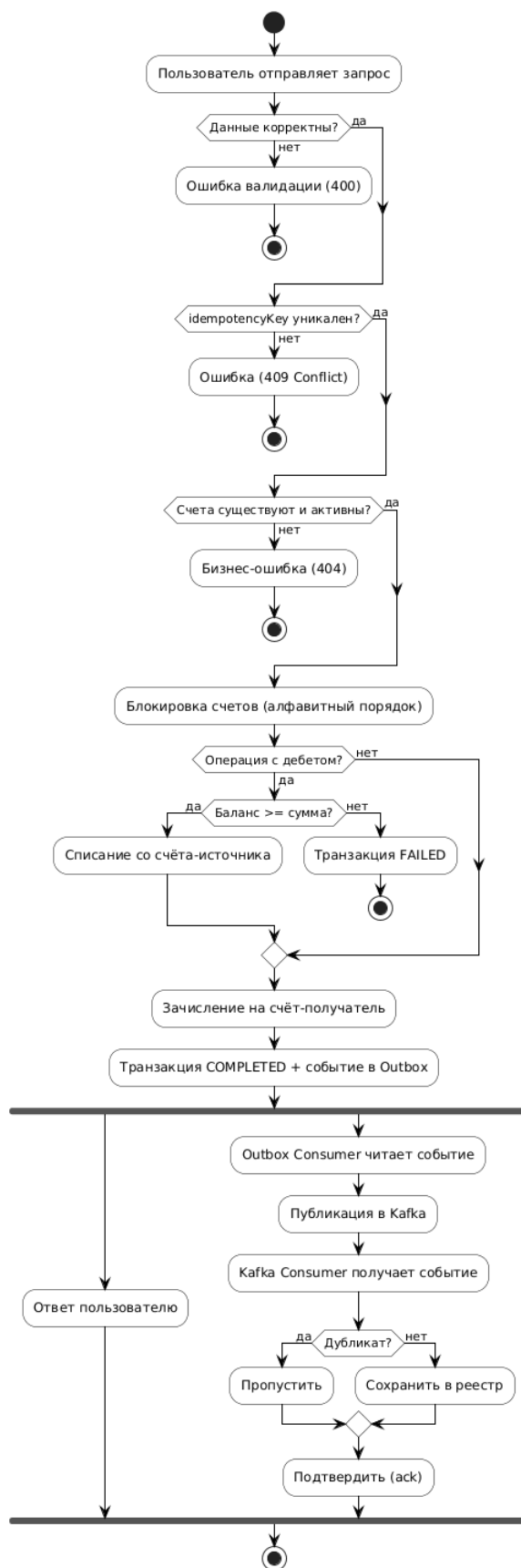


Рисунок 5 – Функциональная модель

На диаграмме показан полный путь транзакции. Сервис core-transaction последовательно проверяет входные данные, ключ идемпотентности, существование и активность счетов, достаточность баланса. При успехе выполняется списание, зачисление, атомарная запись транзакции и события. Далее потоки расходятся: ответ возвращается пользователю немедленно, а Outbox Consumer асинхронно публикует событие в Kafka, откуда core-ledger получает его, проверяет на дубликат и сохраняет в реестр. Для операций без списания (денежный подарок, компенсация) ветвь проверки баланса пропускается.

8.4 Диаграмма компонентов

Диаграмма компонентов отображает физические или логические части системы, их интерфейсы и зависимости. На диаграмме выделены основные компоненты обоих сервисов и внешние системы (рис. 6).

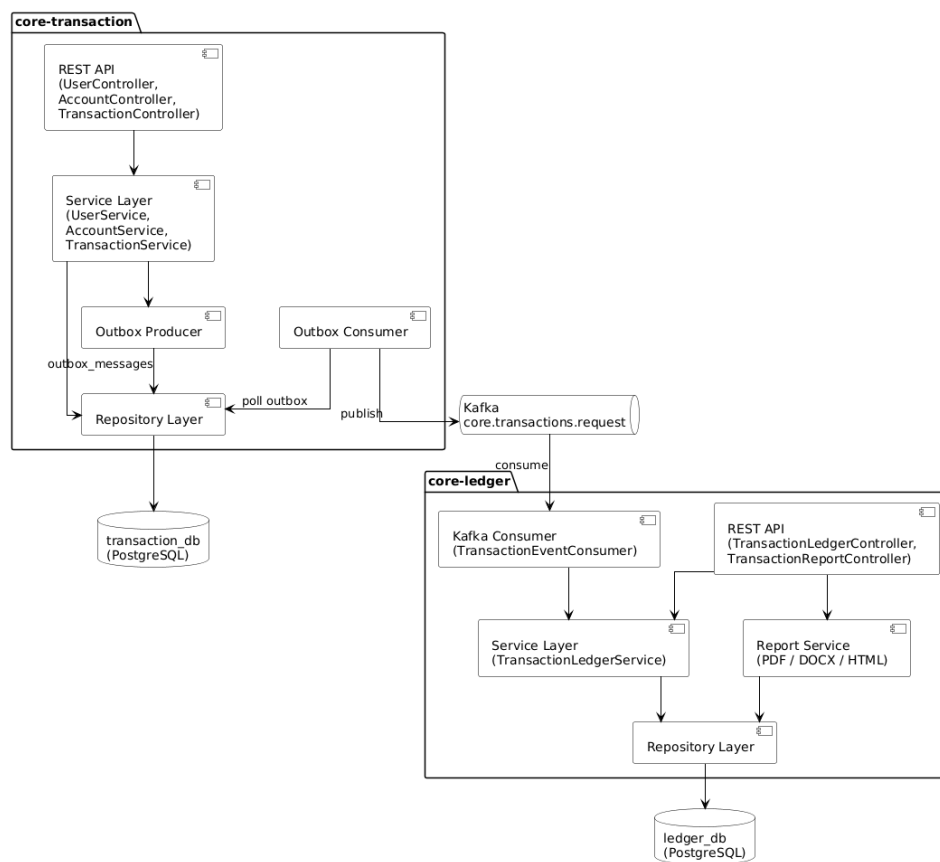


Рисунок 6 – Диаграмма компонентов

Зависимости между компонентами отражают поток данных: клиент обращается к REST API сервиса core-transaction; API использует сервисный слой; сервисный слой вызывает репозитории и Outbox Producer, который сохраняет событие в базу; Outbox Consumer забирает события и публикует их в Kafka; на стороне core-ledger Kafka Consumer получает сообщение и передаёт его в сервисный слой для записи в реестр. REST API сервиса core-ledger обращается к сервисному слою и к Report Service для формирования отчётов.

8.5 Диаграммы последовательности

Для сценария «Перевод на сберегательный счёт» построена диаграмма последовательности с синхронными вызовами и возвратами (рис. 7).

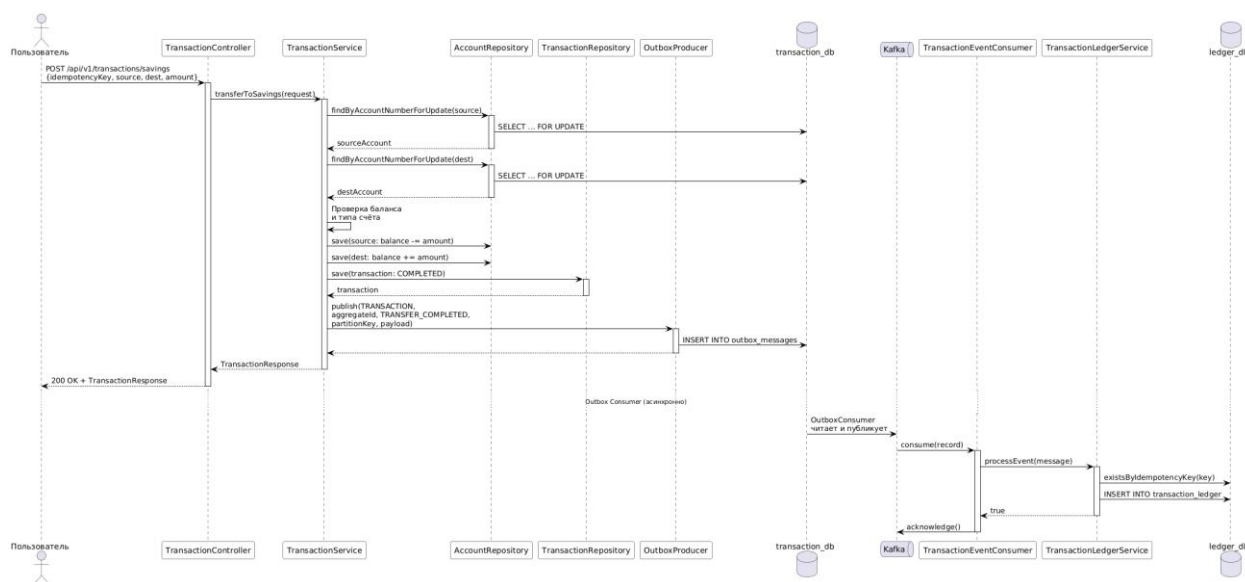


Рисунок 7 – Диаграмма последовательности

Диаграмма показывает синхронный сценарий перевода: контроллер делегирует сервису, сервис в рамках одной БД-транзакции блокирует счета, изменяет балансы, создаёт запись транзакции и сохраняет событие в Outbox. Ответ возвращается пользователю сразу после фиксации транзакции. Асинхронно Outbox Consumer забирает событие и публикует его в Kafka, после чего core-ledger получает сообщение, проверяет дубликат и сохраняет запись в реестр.

8.6 Диаграмма размещения

Диаграмма размещения показывает физическое расположение программных компонентов по узлам инфраструктуры. Выделены основные узлы: клиентское устройство, два сервиса приложений, кластер Kafka и две базы данных (рис. 8).

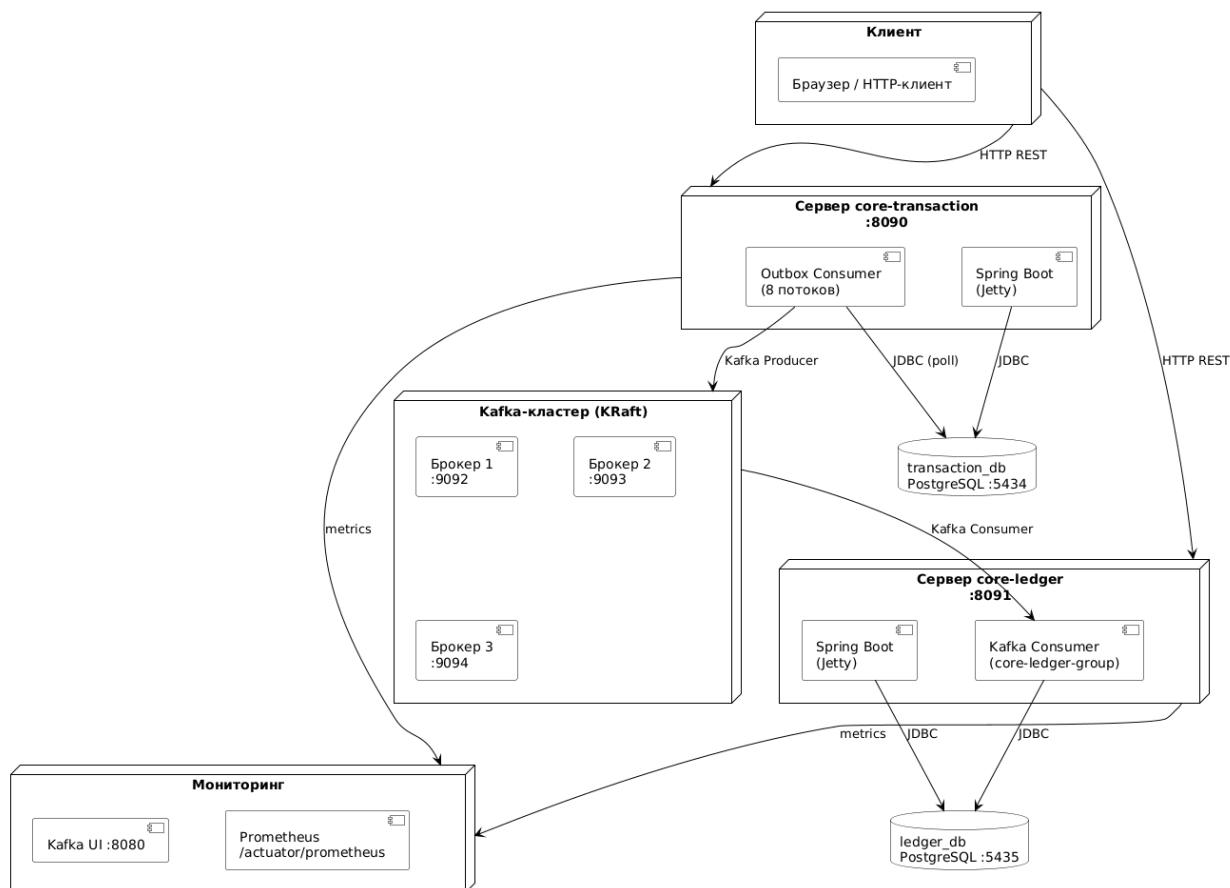


Рисунок 8 – Диаграмма размещения

Клиент обращается к двум сервисам по HTTP. core-transaction работает на порту 8090, взаимодействует с transaction_db через JDBC и публикует события в Kafka-кластер из трёх KRaft-брокеров. core-ledger на порту 8091 потребляет события из Kafka и сохраняет записи в ledger_db. Каждый сервис экспонирует метрики Prometheus на /actuator/prometheus, а Kafka UI на порту 8080 предоставляет веб-интерфейс для мониторинга топиков и потребителей.

8.7 Диаграмма состояний

Диаграмма состояний описывает жизненный цикл финансовой транзакции, outbox события, записи реестра – ключевых сущностей предметной области (рис 9).

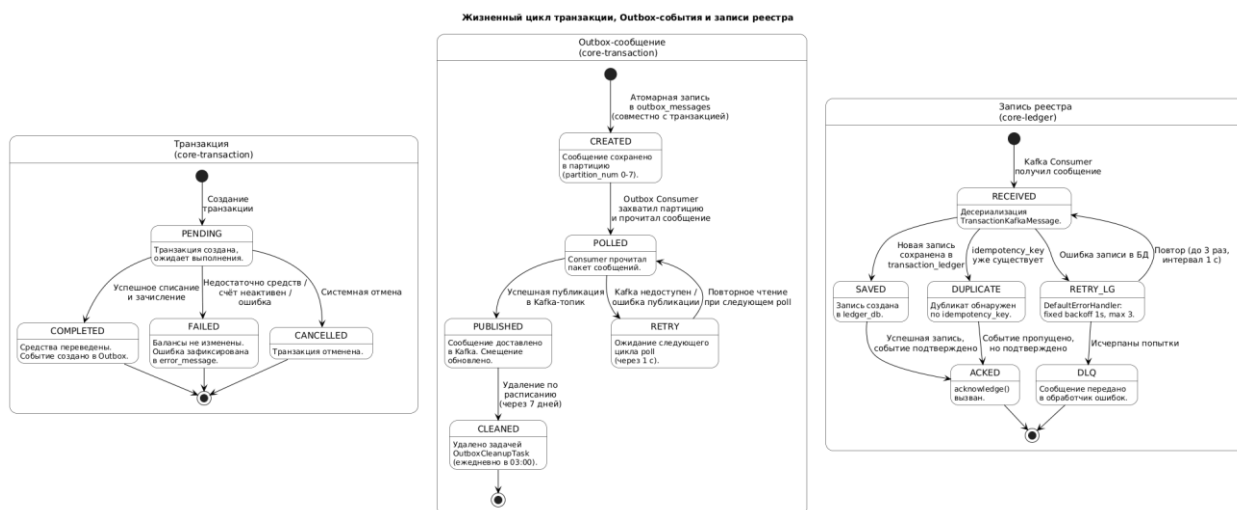


Рисунок 9 – Диаграмма состояний

На диаграмме отражены три жизненных цикла. Транзакция создаётся в состоянии PENDING и переходит в COMPLETED, FAILED или CANCELLED. При статусе COMPLETED атомарно создаётся Outbox-сообщение, которое проходит путь от создания через чтение и публикацию до удаления по расписанию; при ошибке публикации сообщение остаётся в базе и повторно обрабатывается при следующем цикле poll. На стороне core-ledger полученное из Kafka событие проверяется на дубликат, сохраняется в реестр и подтверждается; при ошибке записи применяется политика повторных попыток (до 3 раз с интервалом 1 секунда).

8.8 Диаграммы потоков данных

Диаграмма потоков данных показывает перемещение данных и полезных нагрузок между компонентами при проведении транзакции (рис. 10).

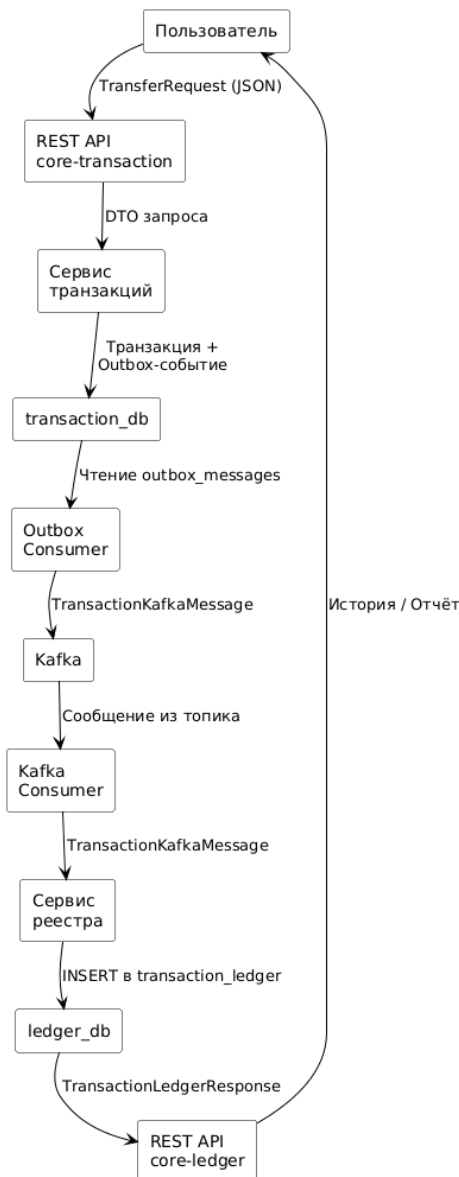


Рисунок 10 – Диаграмма потоков данных

Данные входят в систему как JSON-запрос, обрабатываются сервисом транзакций и фиксируются в transaction_db вместе с Outbox-событием. Outbox Consumer публикует событие в Kafka, core-ledger получает его и сохраняет в реестр. Пользователь запрашивает историю или отчёт через REST API core-ledger.

8.9 Диаграмма прецедентов

Диаграмма прецедентов отображает функциональность системы с точки зрения пользователей (актёров). Выделены основные прецеденты

взаимодействия и связи «включает» (include) и «расширяет» (extend) на рисунке 11.

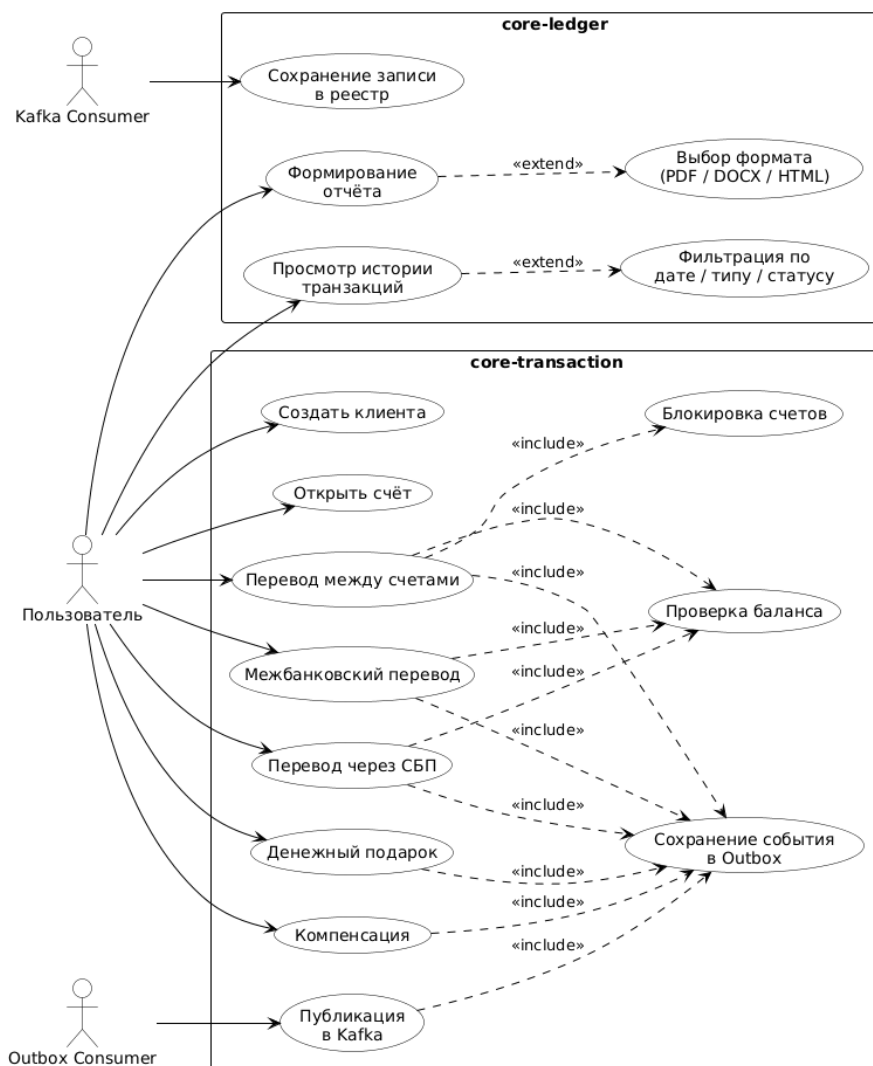


Рисунок 11 – Диаграмма прецедентов

Пользователь может выполнять все основные операции: создание клиента, открытие счёта, проведение шести типов транзакций, просмотр истории и формирование отчётов. Операции с дебетом (перевод, СБП, межбанковский) включают проверку баланса. Все транзакции включают сохранение события в Outbox. Outbox Consumer отвечает за публикацию событий в Kafka. Kafka Consumer на стороне core-ledger сохраняет запись в реестр. Просмотр истории может быть расширен фильтрацией, а формирование отчёта – выбором формата.

8.10 Диаграмма кооперации

Для сценария «Перевод между счетами» построена диаграмма кооперации. Она отражает последовательность взаимодействия между участниками в виде связей между объектами (рис. 12).

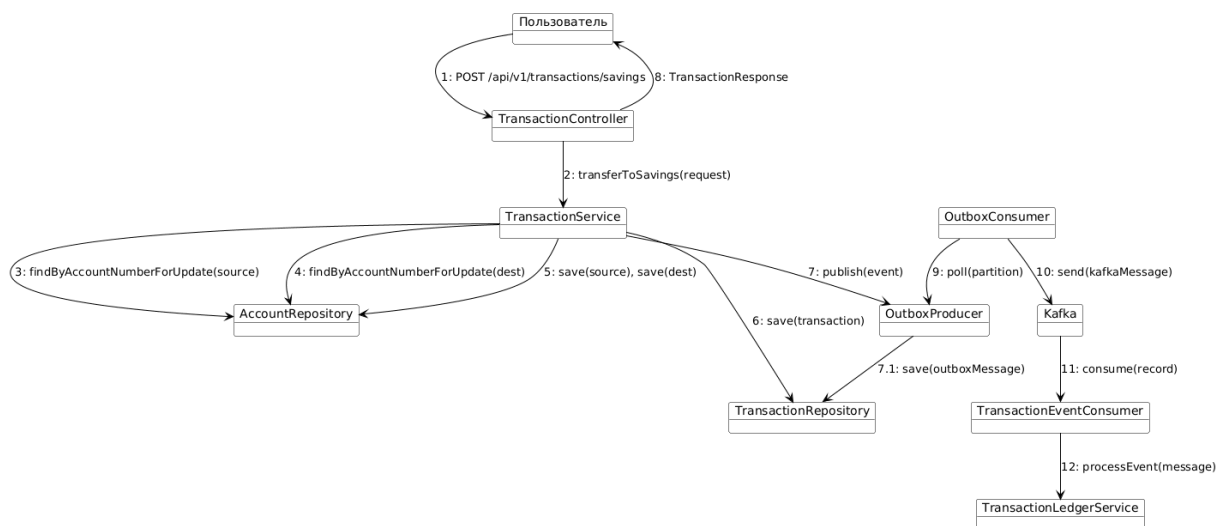


Рисунок 12 – Диаграмма кооперации

На диаграмме пользователь инициирует перевод через контроллер (1), контроллер делегирует сервису (2), сервис блокирует счета (3–4), изменяет балансы (5), сохраняет транзакцию (6) и публикует событие в Outbox (7) – всё в рамках одной БД-транзакции. Ответ возвращается пользователю (8). Асинхронно Outbox Consumer забирает событие (9) и публикует его в Kafka (10), после чего core-ledger получает сообщение (11) и записывает его в реестр (12).

8.11 Диаграмма классов

Диаграмма классов построена для каждого из двух сервисов. Отображены ключевые классы, интерфейсы, атрибуты и методы, а также связи наследования и ассоциации (рис. 13-14).

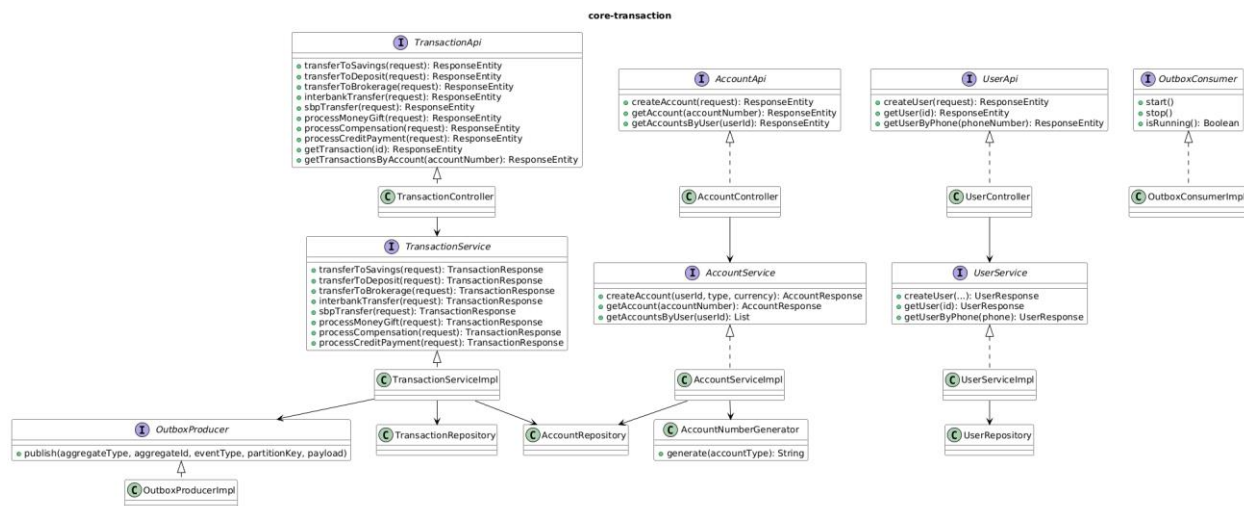


Рисунок 13 – Диаграмма классов сервиса core-transaction

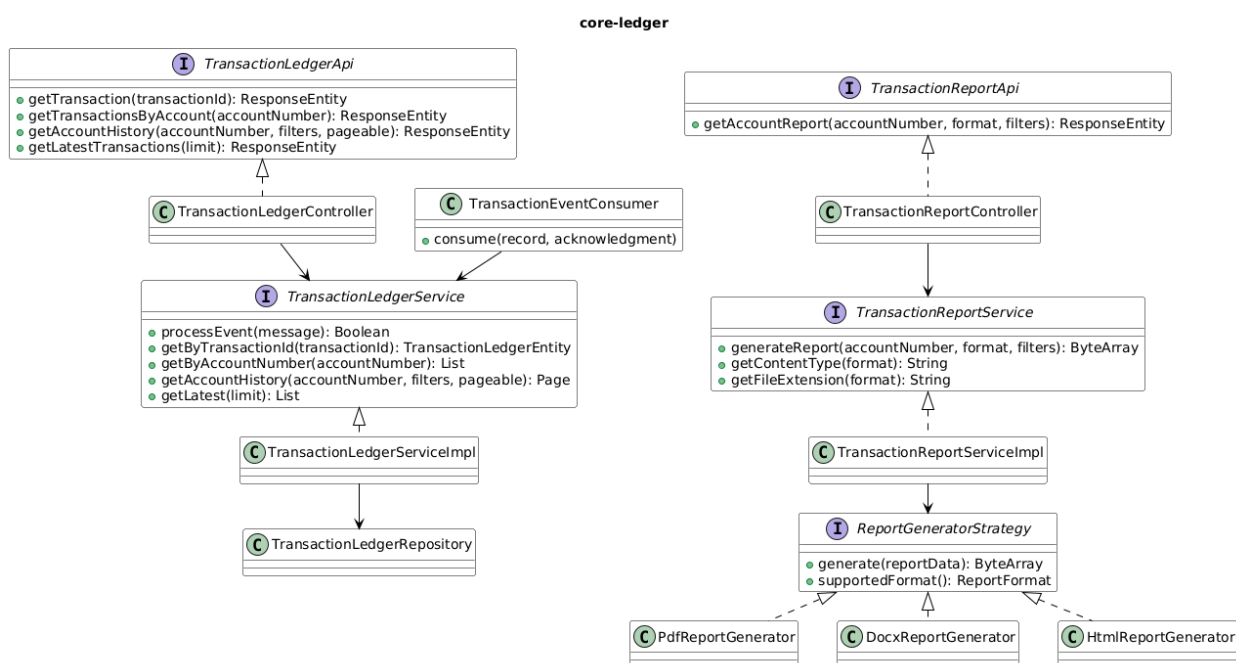


Рисунок 14 – Диаграмма классов сервиса core-ledger

На диаграммах классов видна чёткая структура обоих сервисов: контроллеры реализуют Api-интерфейсы с OpenAPI-аннотациями, сервисные классы реализуют бизнес-интерфейсы. В core-transaction ключевые связи: TransactionServiceImpl зависит от репозитория счетов и транзакций, а также от OutboxProducer. В core-ledger стратегия генерации отчётов реализована через паттерн Strategy: TransactionReportServiceImpl вызывает конкретный генератор (PdfReportGenerator, DocxReportGenerator, HtmlReportGenerator) в зависимости от запрошенного формата.

Диаграмма классов отражает структуру программной системы, реализующей механизм гарантированной доставки сообщений между банковскими сервисами на основе паттерна Transactional Outbox. Основная задача данного механизма заключается в обеспечении надежной передачи событий между микросервисами без риска потери данных и нарушения согласованности бизнес-операций.

Представленная диаграмма классов демонстрирует разделение ответственности между компонентами системы и обеспечивает выполнение ключевого требования банковских информационных систем – гарантированную доставку сообщений при сохранении целостности данных и устойчивости к сбоям.

9 Описание алгоритмов обработки данных

9.1 Описание алгоритмов обработки данных

В системе выделены четыре ключевых алгоритма, которые определяют поведение платформы при обработке финансовых операций и доставке событий между сервисами.

Алгоритм обработки финансовой транзакции (дебет-кредит). При поступлении запроса на перевод сервис core-transaction выполняет следующую последовательность действий в рамках одной БД-транзакции. Сначала проверяется ключ идемпотентности: если транзакция с таким ключом уже существует, возвращается ранее сохранённый результат без повторного выполнения. Затем выполняется поиск и блокировка счетов. Блокировка всегда накладывается в алфавитном порядке по номеру счёта – это исключает взаимную блокировку (deadlock) при одновременных встречных переводах. После блокировки проверяется активность счёта-источника и достаточность средств. Если проверки пройдены, выполняется списание со счёта-источника, зачисление на счёт-получатель, создание записи транзакции со статусом COMPLETED и атомарное сохранение события в таблицу outbox_messages. Для операций без списания (денежный подарок, компенсация) этап проверки баланса и списания пропускается – блокируется и пополняется только счёт-получатель.

Алгоритм работы Outbox Consumer. При запуске сервиса для каждой из восьми партиций создаётся отдельный поток, который с интервалом в 1 секунду выполняет цикл опроса. На каждом цикле поток пытается захватить лизинговую блокировку своей партиции (запись в таблице outbox_partition_locks с временем истечения). Если блокировка захвачена, поток считывает текущее смещение из outbox_consumer_offsets, загружает пакет сообщений (до 50 штук) из outbox_messages начиная с этого смещения, последовательно передаёт каждое сообщение обработчику (TransactionEventHandler), который публикует его в Kafka-топик. После

успешной обработки пакета смещение обновляется. Если обработка одного из сообщений завершается ошибкой, цикл прерывается – смещение обновляется до последнего успешно обработанного сообщения, а оставшиеся будут обработаны при следующем цикле.

Алгоритм идемпотентного потребления Kafka-событий. Kafka Consumer в сервисе core-ledger работает в режиме ручного подтверждения (AckMode.MANUAL_IMMEDIATE). При получении сообщения из топика сервис извлекает idempotencyKey и проверяет его наличие в таблице transaction_ledger. Если запись с таким ключом уже существует – событие считается дубликатом: оно пропускается, но подтверждается (acknowledge), чтобы Kafka не доставляла его повторно. Если ключ не найден – из сообщения создаётся новая запись реестра и сохраняется в базу. После успешного сохранения вызывается acknowledge. При ошибке записи срабатывает DefaultErrorHandler с политикой повторных попыток (до 3 раз с интервалом 1 секунда).

Алгоритм генерации номеров счетов. При создании счёта компонент AccountNumberGenerator формирует уникальный 20-символьный номер. Сначала определяется префикс типа: CHECKING → «1», SAVINGS → «2», DEPOSIT → «3», BROKERAGE → «4». Затем вызывается nextval() соответствующей PostgreSQL-последовательности (seq_account_checking, seq_account_savings и т.д.) через JdbcTemplate. Полученное значение дополняется нулями слева до 19 символов и конкатенируется с префиксом. Использование отдельных последовательностей для каждого типа гарантирует уникальность без конкуренции между типами.

9.2 Блок-схемы алгоритмов обработки данных

Алгоритм обработки финансовой транзакции (дебет-кредит) представлен на рисунке 15:

Алгоритм обработки транзакции (дебет-кредит)

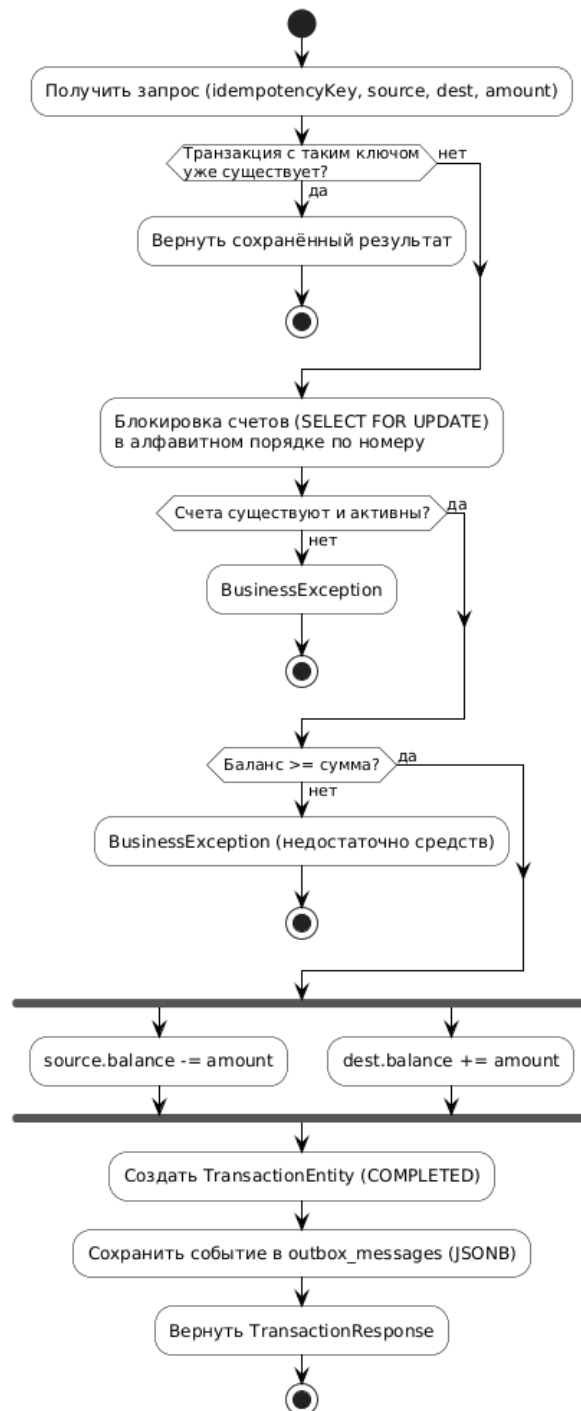


Рисунок 15 – Блок-схема алгоритма обработки транзакции

Алгоритм работы Outbox Consumer (одна партиция) представлен на рисунке 16:

Алгоритм Outbox Consumer (одна партиция)

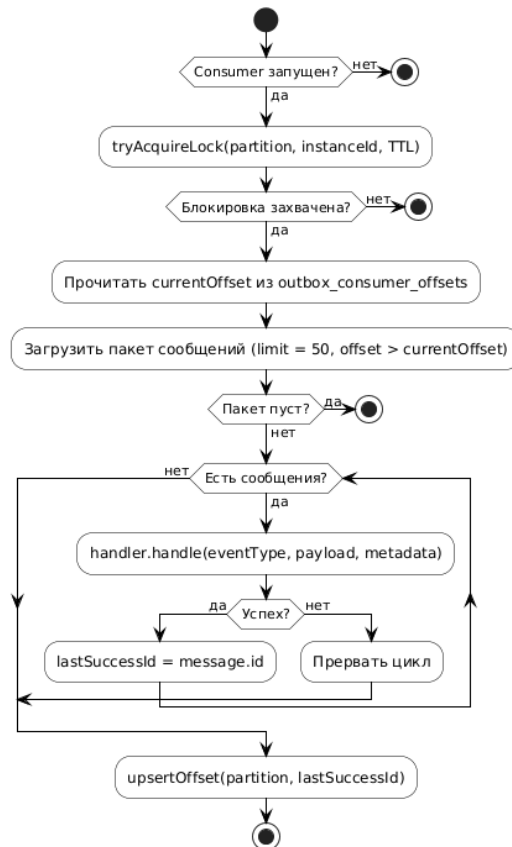


Рисунок 16 – Блок-схема алгоритма Outbox Consumer

Алгоритм идемпотентного потребления Kafka-событий представлен на рисунке 17:

Алгоритм потребления Kafka-событий (core-ledger)

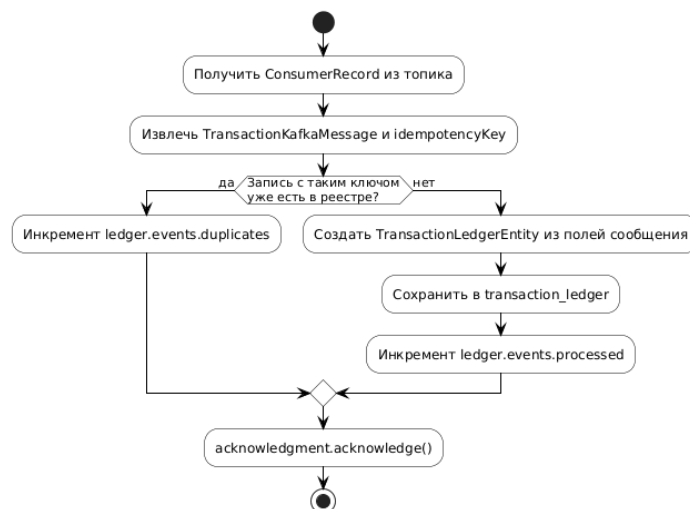


Рисунок 17 – Блок-схема алгоритма потребления Kafka-событий

Алгоритм генерации номера счёта представлен на рисунке 18:

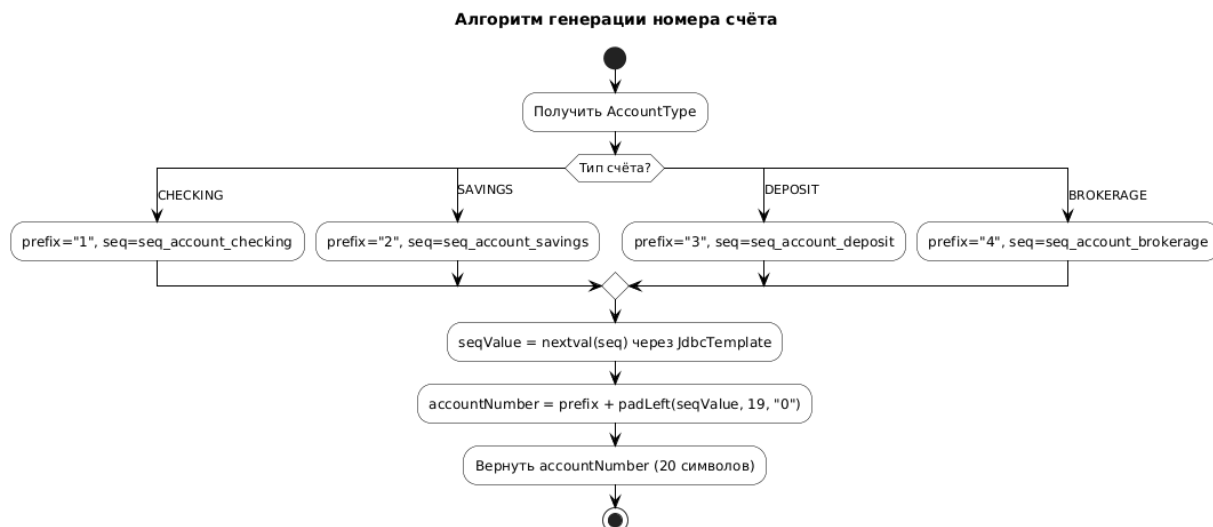


Рисунок 18 – Блок-схема алгоритма генерации номера счёта

На блок-схемах отражены все ключевые ветвления и последовательности действий. Алгоритм обработки транзакции показывает проверку идемпотентности, упорядоченную блокировку счетов, валидацию и атомарное сохранение данных вместе с Outbox-событием. Алгоритм Outbox Consumer демонстрирует лизинговую блокировку, пакетную обработку и обновление смещения. Алгоритм потребления Kafka-событий иллюстрирует идемпотентную дедупликацию по ключу и ручное подтверждение. Алгоритм генерации номера счёта показывает выбор последовательности по типу и формирование 20-символьного идентификатора.

10 Описание реализации программного продукта

10.1 Выбор среды программирования

Реализация системы выполнена на языке Kotlin 1.9.25 с целевой платформой Java 21. Kotlin выбран как основной язык разработки благодаря лаконичному синтаксису, встроенной null-безопасности и полной совместимости с экосистемой Java и Spring.

В качестве фреймворка используется Spring Boot 3.5.12. Встроенный сервер Tomcat заменён на Jetty – он легче по потреблению памяти и лучше подходит для микросервисной архитектуры. Управление зависимостями осуществляется через Gradle (Kotlin DSL).

Таблица 6 – Основные зависимости сервисов

Компонент	Зависимость	Назначение
ORM	Spring Data JPA (Hibernate)	Доступ к данным, репозитории
Миграции	Liquibase	Версионирование схемы БД
Валидация	spring-boot-starter-validation	Jakarta Bean Validation
Сериализация	jackson-module-kotlin	JSON (де)сериализация
Логирование	kotlin-logging-jvm 3.0.5	Структурированное логирование
Документация	springdoc-openapi 2.8.6	Swagger UI на /swagger-ui.html
Метрики	Micrometer + Prometheus	Экспорт на /actuator/prometheus
Сообщения	Spring Kafka	Kafka Producer / Consumer

Сервис core-ledger дополнительно использует OpenPDF 2.0.3 для генерации PDF-отчётов и Apache POI 5.3.0 для генерации DOCX-отчётов.

Для тестирования используются JUnit 5, MockK 1.13.12 (мокирование для Kotlin), Testcontainers (PostgreSQL и Kafka в Docker-контейнерах) и Spring Kafka Test.

10.2 Последовательность разработки прикладной программы

Разработка выполнялась итеративно с привязкой к функциональным блокам:

- проектирование схемы базы данных transaction_db и написание Liquibase-миграций: пользователи, счета, карты, транзакции, последовательности для генерации номеров;
- реализация сервиса core-transaction: REST API для управления пользователями и счетами, бизнес-логика проведения транзакций с пессимистичной блокировкой;
- реализация Transactional Outbox: таблицы outbox_messages (с LIST-секционированием), outbox_consumer_offsets, outbox_partition_locks; компоненты OutboxProducer, OutboxConsumer, TransactionEventHandler;
- настройка Kafka-кластера (3 брокера KRaft) и конфигурация Producer в core-transaction;
- проектирование схемы ledger_db и реализация сервиса core-ledger: Kafka Consumer с ручным подтверждением, идемпотентная запись в реестр, REST API для запроса истории;
- реализация генерации отчётов в форматах PDF, DOCX и HTML через паттерн Strategy;
- написание unit-тестов (MockK) и интеграционных тестов (Testcontainers);
- оформление документации.

10.3 Схема меню приложения

Система не имеет графического пользовательского интерфейса в традиционном понимании – оба сервиса предоставляют REST API. В качестве интерактивного интерфейса для работы с API используется Swagger UI, доступный по адресу /swagger-ui.html на каждом из сервисов. Swagger UI

генерируется автоматически библиотекой springdoc-openapi на основе аннотаций контроллеров.

Структура меню Swagger UI сервиса core-transaction (порт 8090) включает три группы эндпоинтов:

- user-controller – создание клиента, получение по идентификатору и по номеру телефона;
- account-controller – открытие счёта, получение по номеру счёта, получение всех счетов клиента;
- transaction-controller – проведение восьми типов транзакций, получение транзакции по идентификатору, получение транзакций по номеру счёта.

Структура меню Swagger UI сервиса core-ledger (порт 8091) включает две группы:

- transaction-ledger-controller – запрос записи по идентификатору транзакции, по номеру счёта, постраничная история с фильтрами, последние N транзакций;
- transaction-report-controller – формирование отчёта с выбором формата (PDF, DOCX, HTML).

С точки зрения пользователя ключевые сценарии начинаются с операций в core-transaction (создание клиента, открытие счёта, проведение транзакции), после чего результат можно просмотреть через core-ledger (история, отчёт).

10.4 Описание окон приложения

Каждая группа в Swagger UI раскрывается в список эндпоинтов. При выборе конкретного эндпоинта отображается его описание, схема запроса с типами полей и кнопка «Try it out» для интерактивного вызова.

Окно создания клиента (POST /api/v1/users). Содержит форму с полями: firstName (строка, обязательное), lastName (строка, обязательное), patronymic (строка, необязательное), phoneNumber (строка, формат

+XXXXXXXXXXXX), email (строка, необязательное). При успешном создании возвращается JSON с присвоенным идентификатором и датой регистрации.

Окно открытия счёта (POST /api/v1/accounts). Содержит поля: userId (число), accountType (строка: CHECKING, SAVINGS, DEPOSIT, BROKERAGE), currency (строка, по умолчанию RUB). В ответе возвращается сгенерированный номер счёта, тип, баланс и статус.

Окно проведения транзакции (POST /api/v1/transactions/*). Форма зависит от типа операции. Для переводов между счетами – sourceAccountNumber, destinationAccountNumber, amount, idempotencyKey. Для СБП – sourceAccountNumber, recipientPhoneNumber, amount. Для межбанковского перевода – sourceCardNumber, destinationCardNumber. При успешном выполнении возвращается тип, статус, сумма и временные метки.

Окно истории транзакций (GET /api/v1/transactions/account/{accountNumber}/history). Параметры фильтрации: dateFrom, dateTo, type, status, а также page, size, sort. Ответ содержит страничный список записей реестра.

Окно формирования отчёта (GET /api/v1/transactions/account/{accountNumber}/report). Параметр format задаёт формат загрузки. При выполнении браузер скачивает файл с соответствующим расширением.

10.5 Функциональные возможности программного приложения

Функциональные возможности приложения соответствуют утверждённым требованиям:

- создание клиента банка с указанием ФИО, номера телефона и электронной почты;
- открытие счёта выбранного типа с автоматической генерацией уникального 20-символьного номера;
- проведение внутренних переводов между счетами (на сберегательный, депозитный, брокерский);

- проведение перевода через СБП по номеру телефона получателя;
- проведение межбанковского перевода по номерам платёжных карт;
- зачисление средств без списания: денежный подарок и компенсация;
- погашение кредита: списание с расчётного счёта и зачисление на счёт назначения;
- гарантированная доставка событий через Transactional Outbox и Kafka;
- идемпотентная обработка повторных запросов и повторно доставленных событий;
- запрос истории транзакций с фильтрацией по дате, типу операции и статусу;
- формирование отчётов по истории транзакций в форматах PDF, DOCX и HTML;
- экспорт метрик в формате Prometheus для мониторинга.

Перечисленный функционал полностью соответствует утверждённым требованиям, обеспечивая полный жизненный цикл банковской операции – от регистрации клиента до выгрузки отчёта.

10.6 Функции и процедуры обработки данных

Обработка данных выполняется на нескольких уровнях системы.

На уровне контроллеров REST API запросы принимаются типизированными методами. TransactionController делегирует все операции в TransactionService, принимая DTO (TransferRequest, SbpTransferRequest, InterbankTransferRequest, MoneyGiftRequest) и возвращая TransactionResponse. TransactionLedgerController делегирует запросы истории в TransactionLedgerService и преобразует entity в DTO через маппер. TransactionReportController делегирует генерацию отчётов в

TransactionReportService, устанавливая Content-Type и Content-Disposition в заголовках ответа.

На уровне сервисов реализована основная бизнес-логика:

- метод executeDebitCredit() выполняет полный цикл перевода: списание, зачисление, создание записи транзакции и публикация события в Outbox – всё в одной аннотированной @Transactional операции;
- метод executeCreditOnly() аналогичен, но без списания – для денежных подарков и компенсаций;
- метод lockAccountsInOrder() упорядочивает счета по алфавиту и блокирует их через SELECT ... FOR UPDATE;
- метод processEvent() в сервисе реестра реализует идемпотентную запись: проверка существования ключа, создание entity, сохранение;
- метод generateReport() загружает историю, формирует объект ReportData и делегирует нужной стратегии генерации.
- На уровне интеграции с Kafka обработка данных реализована в двух точках:
 - TransactionEventHandler.handle() – десериализация Outbox-события, формирование TransactionKafkaMessage с ключом идемпотентности "\${aggregateId}-\${messageId}" и синхронная отправка в Kafka;
 - TransactionEventConsumer.consume() – получение сообщения, вызов сервиса реестра, инкремент метрик и ручное подтверждение.

Обработка данных разделена по уровням: контроллеры обеспечивают приём и валидацию, сервисный слой реализует бизнес-правила с гарантией атомарности, а интеграционный слой обеспечивает надёжную доставку событий и их идемпотентное потребление.

11 Тестирование программного продукта и отладка

При разработке банковской платформы требуется уделить особое внимание тестированию, поскольку система работает с финансовыми

операциями, где любая ошибка может привести к некорректному движению денежных средств. Для обеспечения качества применена многоуровневая стратегия тестирования, включающую модульные (unit) и интеграционные тесты.

Модульные тесты проверяют логику отдельных компонентов в изоляции – сервисов, контроллеров, мапперов. Для подмены зависимостей я использовал библиотеку MockK, которая хорошо интегрируется с Kotlin и позволяет создавать моки, стабы и верифицировать вызовы методов.

Интеграционные тесты проверяют взаимодействие компонентов с реальной инфраструктурой – базой данных PostgreSQL и брокером сообщений Apache Kafka. Для этого я использовал библиотеку Testcontainers, которая автоматически поднимает Docker-контейнеры с необходимыми сервисами перед запуском тестов и останавливает их после завершения (рис. 19-20).

Test Summary

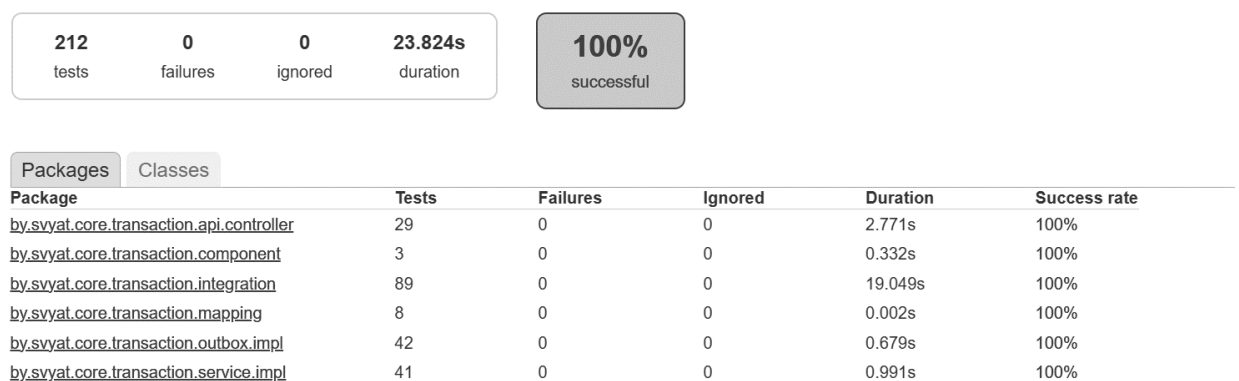


Рисунок 19 - Результат запуска тестов сервиса core-transaction

Test Summary

50 tests	0 failures	0 ignored	15.577s duration	100% successful
-------------	---------------	--------------	---------------------	--------------------

Packages	Classes					
Package		Tests	Failures	Ignored	Duration	Success rate
by.svyat.core.ledger.consumer		2	0	0	0.697s	100%
by.svyat.core.ledger.integration		16	0	0	13.991s	100%
by.svyat.core.ledger.mapping		1	0	0	0.001s	100%
by.svyat.core.ledger.service.impl		18	0	0	0.575s	100%
by.svyat.core.ledger.service.report		13	0	0	0.313s	100%

Рисунок 20 - Результат запуска тестов сервиса core-ledger

По результатам тестирования все тесты обоих сервисов завершились успешно. В сервисе core-transaction выполнен 21 тестовый класс, покрывающий все типы банковских операций – переводы между счетами, межбанковские переводы, переводы через СБП, денежные подарки, компенсации. Интеграционные тесты подтвердили корректность работы с реальной базой данных PostgreSQL 16, включая генерацию уникальных номеров счетов и карт через последовательности, механизм идемпотентности, пессимистичную блокировку счетов и запись событий в Outbox-таблицу.

В сервисе core-ledger выполнено 12 тестовых классов. Интеграционные тесты подтвердили корректность полного цикла обработки событий: от получения сообщения из Kafka-топика до его сохранения в таблице transaction_ledger с проверкой идемпотентности. Также проверена работа REST API для получения истории операций с фильтрацией и пагинацией, а также генерация отчётов в форматах PDF, DOCX и HTML.

Ошибок и сбоев в ходе тестирования выявлено не было. Все проверки – валидация входных данных, обработка граничных случаев (недостаточно средств, несуществующий счёт, дублирующий запрос), корректность балансов после операций – пройдены успешно, что подтверждает готовность системы к эксплуатации.

12 Проектирование технической информации к ПО

12.1 Требования к окружению и подключение

В рамках работы используется локальное окружение: оба микросервиса, PostgreSQL и Kafka предполагаются доступными на машине разработчика в Docker-контейнерах с пробросом портов на localhost. Промышленное или облачное развёртывание здесь не детализируется.

Для локального запуска нужны установленные Java 21 (JDK) и Docker с Docker Compose. Порядок запуска инфраструктуры:

- kafka-кластер – из корневой директории проекта: `docker-compose up -d`. Поднимаются три KRaft-брокера (порты 9092, 9093, 9094) и Kafka UI (порт 8080).
- бд core-transaction – из директории core-transaction: `docker-compose up -d`. Поднимается PostgreSQL 16 на порту 5434, база `transaction_db`.
- бд core-ledger – из директории core-ledger: `docker-compose up -d`. Поднимается PostgreSQL 16 на порту 5435, база `ledger_db`.

После запуска инфраструктуры сервисы стартуют командой `./gradlew bootRun` из своих директорий. Liquibase автоматически применяет миграции при первом запуске.

Основные параметры сервиса core-transaction задаются в файле `core-transaction/src/main/resources/application.yaml`:

- `server.port=8090` – порт HTTP-сервера;
- `spring.datasource.url=jdbc:postgresql://localhost:5434/transaction_db` – подключение к PostgreSQL;
- `spring.kafka.bootstrap-servers=localhost:9092,localhost:9093,localhost:9094` – адреса Kafka-брокеров;
- `outbox.partition-count=8` – количество партиций Outbox-таблицы;
- `outbox.poll-interval-ms=1000` – интервал опроса партиций (мс);
- `outbox.batch-size=50` – размер пакета сообщений при опросе;
- `outbox.consumer-group=core-transaction` – имя группы потребителя;

- `outbox.lock-ttl-seconds=30` – время жизни lease-блокировки партии (с);
- `outbox.cleanup-retention-days=7` – срок хранения обработанных сообщений (дней).

Основные параметры сервиса `core-ledger` задаются в файле `core-ledger/src/main/resources/application.yaml`:

- `server.port=8091` – порт HTTP-сервера;
- `spring.datasource.url=jdbc:postgresql://localhost:5435/ledger_db` – подключение к PostgreSQL;
- `spring.kafka.bootstrap-servers=localhost:9092,localhost:9093,localhost:9094` – адреса Kafka-брокеров;
- `kafka.consumer.topic=core.transactions.request` – имя Kafka-топика для потребления;
- `kafka.consumer.group-id=core-ledger-group` – группа потребителей Kafka.

Параметры подключения к PostgreSQL (логин, пароль) задаются в тех же файлах через поля `spring.datasource.username` и `spring.datasource.password`. Метрики обоих сервисов доступны по пути `/actuator/prometheus`.

12.2 Основные REST endpoint

REST API реализовано в обоих сервисах и использует префикс `/api/v1`. Интерактивная документация доступна через Swagger UI: `http://localhost:8090/swagger-ui.html` (`core-transaction`) и `http://localhost:8091/swagger-ui.html` (`core-ledger`).

Endpoint сервиса `core-transaction` (запись):

- `POST /api/v1/users` – регистрация нового пользователя;
- `GET /api/v1/users/{id}` – получение пользователя по идентификатору;
- `GET /api/v1/users/phone/{phoneNumber}` – поиск пользователя по номеру телефона;

- POST /api/v1/accounts – открытие нового банковского счёта;
- GET /api/v1/accounts/{accountNumber} – получение счёта по номеру;
- GET /api/v1/accounts/user/{userId} – получение всех счетов пользователя;
- POST /api/v1/transactions/savings – перевод на сберегательный счёт;
- POST /api/v1/transactions/deposit – перевод на депозитный счёт;
- POST /api/v1/transactions/brokerage – перевод на брокерский счёт;
- POST /api/v1/transactions/interbank – межбанковский перевод по номеру карты;
- POST /api/v1/transactions/sbp – перевод через СБП по номеру телефона;
- POST /api/v1/transactions/gift – денежный подарок (зачисление без списания);
- POST /api/v1/transactions/compensation – компенсация (зачисление без списания);
- POST /api/v1/transactions/credit-payment – платёж по кредиту;
- GET /api/v1/transactions/{id} – получение транзакции по идентификатору;
- GET /api/v1/transactions/account/{accountNumber} – получение транзакций по номеру счёта.

Во всех POST-эндпоинтах транзакций в теле запроса передаётся ключ идемпотентности (idempotencyKey), который предотвращает повторное выполнение операции при дублирующем запросе.

Endpoint сервиса core-ledger (чтение):

- GET /api/v1/transactions/{transactionId} – получение записи по идентификатору исходной транзакции;

- GET /api/v1/transactions/account/{accountNumber} – все записи по номеру счёта;
- GET /api/v1/transactions/account/{accountNumber}/history – история операций с фильтрацией по дате, типу и статусу, с пагинацией;
- GET /api/v1/transactions/latest – последние N транзакций (по умолчанию 20);
- GET /api/v1/transactions/account/{accountNumber}/report – генерация отчёта в формате PDF, DOCX или HTML.

Ошибки возвращаются в едином формате ErrorResponse с полями timestamp, status, message и path

12.3 Формат сообщений для Kafka

При успешном завершении транзакции Outbox-потребитель публикует в топик core.transactions.request сообщение типа TransactionKafkaMessage, сериализованное в JSON.

Сообщение содержит следующие поля:

- idempotencyKey – ключ идемпотентности в формате "{aggregateId}-{messageId}" (String);
- eventType – тип события (например, TRANSFER_COMPLETED);
- payload – тело события с данными транзакции;
- metadata – метаданные события.
- Тело события (payload) содержит:
 - transactionId – идентификатор исходной транзакции (Long);
 - type – тип операции;
 - sourceAccountNumber – номер счёта списания (null для операций без списания);
 - destinationAccountNumber – номер счёта зачисления;
 - amount – сумма операции (BigDecimal);
 - currency – валюта, по умолчанию RUB;

- status – статус транзакции (COMPLETED или FAILED).
- Метаданные (metadata) содержат:
- aggregateId – идентификатор агрегата, используется как ключ Kafka-сообщения для гарантии порядка;
- partitionKey – ключ партиционирования (номер счёта);
- sourceMessageId – идентификатор исходного Outbox-сообщения;
- createdAt – время создания события в формате ISO 8601.

Consumer в core-ledger извлекает idempotencyKey и проверяет наличие записи с таким ключом в таблице transaction_ledger. Если запись существует – сообщение считается дубликатом и пропускается. Если нет – данные из payload и metadata мапятся в новую запись леджера и сохраняются в базу данных. После успешного сохранения вызывается acknowledgment.acknowledge(), что подтверждает сообщение на стороне Kafka.

12.4 Реализация паттерна Transactional Outbox

Для надёжной доставки событий между сервисами я реализовал паттерн Transactional Outbox. Вместо прямой отправки в Kafka событие записывается в таблицу outbox_messages в рамках той же транзакции базы данных, что и бизнес-операция. Если транзакция откатывается – откатывается и событие. Фоновый процесс (Outbox Consumer) периодически читает новые сообщения из таблицы и публикует их в Kafka.

Таблица outbox_messages использует LIST-партиционирование PostgreSQL по полю partition_num – создано 8 партиций (от outbox_messages_p0 до outbox_messages_p7). Номер партиции вычисляется по формуле $\text{abs}(\text{partitionKey.hashCode()}) \% 8$, где partitionKey – номер счёта. Это гарантирует, что все события одного счёта обрабатываются последовательно в одной партиции.

Outbox Consumer при старте приложения создаёт пул из 8 потоков – по одному на партицию. Каждый поток с интервалом 1 секунда выполняет цикл:

захват lease-блокировки (TTL 30 секунд) → чтение пакета до 50 сообщений → маршрутизация к обработчику → обновление смещения. Lease-блокировки хранятся в таблице `outbox_partition_locks` и предотвращают обработку одной партии несколькими экземплярами сервиса.

Очистка старых сообщений выполняется автоматически: ежедневно в 3:00 компонент `OutboxCleanupTask` удаляет сообщения старше 7 дней и истёкшие блокировки.

12.5 Мониторинг

Для мониторинга работы системы используется библиотека `Micrometer` с экспортом метрик в формат `Prometheus`. Метрики доступны по пути `/actuator/prometheus` на каждом сервисе.

Бизнес-метрики `core-transaction`:

- `outbox.producer.published` – счётчик опубликованных событий (теги: тип агрегата, тип события);
- `outbox.consumer.processed` – счётчик обработанных сообщений (теги: тип агрегата, тип события, статус);
- `outbox.consumer.lag` – задержка обработки сообщений по партициям;
- `outbox.consumer.lock.acquired` – результат захвата блокировки (успех/неудача);
- `outbox.consumer.errors` – количество ошибок обработки по партициям.

Бизнес-метрики `core-ledger`:

- `ledger.events.processed` – счётчик обработанных событий (теги: тип события, статус);
- `ledger.events.duplicates` – счётчик пропущенных дубликатов.

Помимо бизнес-метрик, `Spring Boot Actuator` автоматически собирает метрики HTTP-запросов, пула соединений `HikariCP` и JVM. Для визуального

мониторинга Kafka-кластера используется Kafka UI, доступный по адресу <http://localhost:8080>.

13 Безопасность и экологичность проекта

13.1 Значение и задачи безопасности жизнедеятельности

Эксплуатация разработанной событийно-управляемой банковской платформы связана с необходимостью работы за персональным компьютером или ноутбуком, обеспечения сетевого взаимодействия микросервисов платформы между собой и с брокером сообщений, а также обслуживания серверов баз данных.

Значительная часть рабочего времени проходит в положении сидя перед экраном при ограниченной двигательной активности, что в сочетании с высокой концентрацией внимания повышает зрительную и нервно-эмоциональную нагрузку на работника.

Особенности труда при работе с программными и аппаратными средствами платформы связаны с воздействием ряда опасных и вредных производственных факторов:

- электромагнитные поля, источником которых служат персональные компьютеры, серверное и сетевое оборудование;
- недостаточная либо неравномерная освещённость рабочей зоны, повышающая утомляемость органов зрения;
- повышенный уровень шума, создаваемый системами охлаждения серверного и рабочего оборудования;
- опасность поражения электрическим током при нарушении изоляции или неисправности оборудования;
- нерациональная организация рабочего места, увеличивающая нагрузку на опорно-двигательный аппарат;
- монотонность и зрительное напряжение, обусловленные длительным наблюдением за экраном монитора.

Значение безопасности жизнедеятельности при проектировании и разработке банковской платформы состоит в своевременном выявлении указанных факторов и снижении их влияния на разработчика до допустимых

уровней. Своевременный учёт этих требований сопровождал все этапы проектирования системы.

Основные задачи безопасности жизнедеятельности в рамках выполненной работы сведены к следующему:

- выявление и оценка значимости опасных и вредных факторов рабочей среды;
- снижение воздействия этих факторов средствами организационного и технического характера;
- предупреждение профессиональных заболеваний и производственного травматизма;
- создание условий труда, отвечающих требованиям электробезопасности и пожарной безопасности;
- готовность персонала к действиям в условиях возможных чрезвычайных ситуаций.

Решение перечисленных задач опирается на действующие нормативные документы:

- Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ;
- ГОСТ 12.0.003-2015 «ССБТ. Опасные и вредные производственные факторы. Классификация»;
- ГОСТ 12.1.019-2017 «ССБТ. Электробезопасность. Общие требования и номенклатура видов защиты»;
- СП 2.2.3670-20 «Санитарно-эпидемиологические требования к условиям труда»;
- СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания».

13.2 Анализ условий труда и мероприятия по защите от воздействия вредных производственных факторов

Основная работа с разработанной платформой выполняется разработчиками программного обеспечения и системными администраторами. Рабочий процесс включает длительное взаимодействие с персональным компьютером или ноутбуком при написании кода, отладке программного обеспечения, настройке брокера сообщений и мониторинге работы системы и проходит в рабочем положении сидя.

В связи с характером выполняемых работ значительное влияние оказывают такие производственные факторы, как освещение, электромагнитные поля, шум систем охлаждения серверного оборудования и пыль. Меры по снижению влияния вредных факторов приведены далее:

- освещение на предприятии организовано в соответствии с ГОСТ Р 55710-2013 «Освещение рабочих мест внутри зданий. Нормы и методы измерений»;
- допустимые значения электромагнитных полей регламентированы с СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания»;
- рекомендованы регулярное проведение влажной уборки и вентиляции помещения в соответствии с ГОСТ 12.1.005-88 «Система стандартов безопасности труда. Общие санитарно-гигиенические требования к воздуху рабочей зоны»;
- рабочие места системных администраторов оборудованы в соответствии с ГОСТ Р 50923-96 «Дисплеи. Рабочее место оператора. Общие эргономические требования и требования к производственной среде. Методы измерения».

13.3 Обеспечение электробезопасности

Активная работа с платформой проходит в двух типах помещений – в кабинете разработки и сопровождения, расположенном в офисном помещении, а также в серверном помещении, где функционируют серверная часть системы, серверы баз данных и брокер сообщений.

В соответствии с «Правилами устройства электроустановок (ПУЭ)» эти помещения классифицированы как помещения без повышенной опасности, так как в них отсутствуют токопроводящие полы, температура и влажность воздуха не повышены и устранена возможность одновременного прикосновения к металлическим корпусам электрооборудования и заземляющим конструкциям.

Основными местами накопления статического электричества в этих помещениях являются пластиковые корпуса оборудования, кабельная изоляция и потоки воздуха от систем кондиционирования серверной.

Приняты меры для дальнейшего снижения опасности поражения током в соответствии с ГОСТ 12.1.019-2017 «Система стандартов безопасности труда. Электробезопасность. Общие требования и номенклатура видов защиты»:

- провода изолированы и размещены вне зоны досягаемости частями тела сотрудников;
- оборудование установлено вдали от элементов теплоснабжения на сухие поверхности;
- всё электрооборудование заземлено, для серверной организован отдельный заземляющий контур;
- введены предупредительная сигнализация и защитное отключение, срабатывающие в случае возникновения короткого замыкания и других аварийных ситуаций.

12.4 Пожарная безопасность

В условиях эксплуатации разработанной платформы работа выполняется в помещениях, в которых основными пожароопасными веществами являются твёрдые горючие материалы в составе технического оборудования – изоляция проводов и компоненты системных блоков, мониторов и серверного оборудования.

В соответствии с федеральным законом «Технический регламент о требованиях пожарной безопасности» от 22.07.2008 N 123-ФЗ и «Правилами устройства электроустановок (ПУЭ)» проведена классификация помещений по пожароопасности. Эта классификация приведена в таблице 7:

Таблица 7 - Категории помещений и класс зон по пожароопасности

Вид помещения	Категория помещения по «Техническому регламенту о требованиях пожарной безопасности»	Класс зоны по ПУЭ
Кабинет разработки и сопровождения	B1	П-IIa
Серверное помещение	B1	П-IIa

Для обнаружения пожара на потолках каждого помещения установлены дымовые пожарные извещатели, входящие в состав автоматической пожарной сигнализации. В качестве первичного средства пожаротушения каждое помещение оборудовано хладоновым огнетушителем, расположенным в наиболее легкодоступной его части. Применение хладоновых составов предпочтительно для серверной, поскольку они не повреждают электронное оборудование.

Для повышения пожарной безопасности рекомендовано обеспечение исправного состояния технического оборудования и электропроводки, а также соблюдение допустимой нагрузки на сеть электропитания серверного оборудования.

12.5 Безопасность жизнедеятельности в чрезвычайных ситуациях

При работе на предприятии среди наиболее вероятных чрезвычайных ситуаций, с которыми могут столкнуться сотрудники, выделены следующие, обусловленные внутренними и внешними причинами:

- возникновение пожара в результате неисправности электропроводки и технического оборудования и нарушение правил пожарной безопасности сотрудниками. Установлен порядок действий при пожаре, включающий в себя извещение о возгорании, меры по ограничению развития пожара, эвакуацию сотрудников, оказание первой помощи и применение средств первичного тушения пожара по возможности;
- попадание в здание предприятия осколков беспилотных летательных аппаратов (БПЛА). Установлен порядок действий в соответствии с рекомендациями МЧС в случае извещения об опасности БПЛА, в соответствии с которым сотрудники покидают рабочие места перемещаются в наименее опасные пространства – первый этаж здания или промежутки между несущих стен.

В связи с потенциально массовым характером чрезвычайных ситуаций приняты широкие меры по снижению их последствий. При найме сотрудников на работу проводится инструктаж по действиям во время чрезвычайных ситуаций. Каждый этаж оборудован схемой эвакуации, отображающей безопасный путь выхода из здания, а также аптечкой с медицинскими средствами, позволяющими оказать первую помощь пострадавшим.

Предложенные меры достаточны для обеспечения безопасности жизнедеятельности персонала, в частности системных администраторов и других сотрудников, работающих с разработанным ПО для мониторинга деятельности персонала. Учтены вероятные факторы риска и предложены меры по их предупреждению и минимизации их последствий.

Заключение

В выпускной квалификационной работе спроектирована и реализована полноценная событийно-ориентированная банковская платформа, состоящая из двух микросервисов на стеке Kotlin, Spring Boot, PostgreSQL и Apache Kafka. Все материалы систематизированы в единый документ: от описания предметной области и технического задания до проектирования базы данных, архитектуры, набора UML-диаграмм, алгоритмов обработки транзакций и справочного раздела с параметрами запуска.

Сформулированы функциональные и нефункциональные требования к системе, спроектирована модель данных с ER-схемой и схемой миграций. Спроектировано разделение ответственности по паттерну CQRS: сервис core-transaction отвечает за запись и проведение восьми типов банковских операций (внутренние переводы, переводы по СБП, межбанковские переводы по номерам карт, зачисления и др.), а сервис core-ledger – за чтение, ведение истории операций, фильтрацию и генерацию отчётов в форматах PDF, DOCX и HTML.

Для надёжной доставки событий между сервисами спроектирован паттерн Transactional Outbox с LIST-партиционированной таблицей, блокировками и пакетной обработкой, что гарантирует атомарность записи бизнес-данных и событий, а также сохранение порядка обработки операций в рамках одного счёта.

Особое внимание при проектировании уделено обеспечению корректности финансовых операций. Спроектирована идемпотентность через уникальные ключи на обоих сервисах, исключающая повторное выполнение одной и той же операции. Определён механизм пессимистичной блокировки счетов в фиксированном порядке, предотвращающий взаимные блокировки при параллельных встречных переводах. Ограничение неотрицательного баланса вынесено на уровень базы данных. Спроектирована схема подтверждения Kafka-сообщений только после успешной записи в БД core-

ledger, что обеспечивает доставку «не менее одного раза» в сочетании с идемпотентной обработкой.

В дальнейшей перспективе архитектура платформы может быть расширена: подключение визуализации метрик через Grafana, добавление кэширования часто запрашиваемых данных средствами Redis, реализация нотификаций пользователей о статусе операций, а также подготовка Kubernetes-манифестов для промышленного развёртывания.

Разработанная система является учебным проектом и не представляет архитектуру или код реальных банковских систем.

Список использованных источников

1. Баланов А. Н. Построение микросервисной архитектуры и разработка высоконагруженных приложений [Электронный ресурс]. – СПб.: Лань, 2025. – 244 с. URL: <https://reader.lanbook.com/book/456920> (дата обращения: 29.03.2026).
2. Алпатов А. Н., Рогов И. Е. Архитектура, проектирование и разработка программных средств [Электронный ресурс]. – М.: МИРЭА – Российский технологический университет, 2023. – 120 с. URL: <https://reader.lanbook.com/book/386189> (дата обращения: 29.03.2026).
3. Richardson C. Pattern: Transactional Outbox [Электронный ресурс]. URL: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обращения: 28.12.2025).
4. Richardson C. Pattern: Microservice Architecture [Электронный ресурс]. URL: <https://microservices.io/patterns/microservices.html> (дата обращения: 28.12.2025).
5. Spring Boot Reference Documentation [Электронный ресурс]. URL: <https://docs.spring.io/spring-boot/documentation.html> (дата обращения: 29.03.2026).
6. Apache Kafka Documentation [Электронный ресурс]. URL: <https://kafka.apache.org/documentation/> (дата обращения: 29.03.2026).
7. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 29.03.2026).
8. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 29.03.2026).

Приложение А

Листинг программы

А.1 Конфигурация и сборка

```
plugins {  
    kotlin("jvm") version "1.9.25"  
    kotlin("plugin.spring") version "1.9.25"  
    id("org.springframework.boot") version "3.5.12"  
    id("io.spring.dependency-management") version "1.1.7"  
    kotlin("plugin.jpa") version "1.9.25"  
}  
  
group = "by.svyat"  
version = "0.0.1-SNAPSHOT"  
description = "core-transaction"  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(21)  
    }  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-  
actuator")  
    implementation("org.springframework.boot:spring-boot-starter-data-  
jpa")  
    implementation("org.springframework.boot:spring-boot-starter-web")  
    {  
        exclude(module = "spring-boot-starter-tomcat")  
    }  
    implementation("org.springframework.boot:spring-boot-starter-  
jetty")  
    implementation("org.springframework.boot:spring-boot-starter-  
validation")  
    implementation("com.fasterxml.jackson.module:jackson-module-  
kotlin")  
    implementation("org.jetbrains.kotlin:kotlin-reflect")  
    implementation("org.liquibase:liquibase-core")  
    implementation("org.postgresql:postgresql")  
    implementation("org.springframework.boot:spring-boot-starter-  
security")  
    implementation("org.springframework.kafka:spring-kafka")  
    implementation("io.github.micrutils:kotlin-logging-jvm:3.0.5")  
}
```

```

        implementation("org.springdoc:springdoc-openapi-starter-webmvc-ui:2.8.6")
        implementation("io.jsonwebtoken:jjwt-api:0.12.6")
        runtimeOnly("io.jsonwebtoken:jjwt-impl:0.12.6")
        runtimeOnly("io.jsonwebtoken:jjwt-jackson:0.12.6")
        runtimeOnly("io.micrometer:micrometer-registry-prometheus")

        testImplementation("org.springframework.boot:spring-boot-starter-test")
        testImplementation("org.springframework.security:spring-security-test")
        testImplementation("org.springframework.boot:spring-boot-testcontainers")
        testImplementation("org.jetbrains.kotlin:kotlin-test-junit5")
        testImplementation("io.mockk:mockk:1.13.12")
        testImplementation("org.testcontainers:junit-jupiter")
        testImplementation("org.testcontainers:postgresql")
        testImplementation("org.testcontainers:kafka")
        testImplementation("org.springframework.kafka:spring-kafka-test")
        testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    }

    kotlin {
        compilerOptions {
            freeCompilerArgs.addAll("-Xjsr305=strict")
        }
    }

    allOpen {
        annotation("jakarta.persistence.Entity")
        annotation("jakarta.persistence.MappedSuperclass")
        annotation("jakarta.persistence.Embeddable")
    }

    tasks.withType<Test> {
        useJUnitPlatform()
    }

    server:
        port: 8090
        shutdown: graceful

    spring:
        application:
            name: core-transaction
        autoconfigure:
            exclude:
                -
org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration

```



```

-
org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAutoConfiguration
main:
  banner-mode: off
datasource:
  url: jdbc:postgresql://localhost:5434/transaction_db
  username: ${DB_USERNAME:postgres}
  password: ${DB_PASSWORD:postgres}
  driver-class-name: org.postgresql.Driver
hikari:
  pool-name: core-transaction-pool
  minimum-idle: 5
  maximum-pool-size: 20
  idle-timeout: 300000
  max-lifetime: 1200000
  connection-timeout: 10000
  keepalive-time: 60000
  leak-detection-threshold: 30000
  validation-timeout: 5000
jpa:
  hibernate:
    ddl-auto: validate
  open-in-view: false
  show-sql: false
  properties:
    hibernate:
      format-sql: false
    jdbc:
      time_zone: Europe/Moscow
kafka:
  bootstrap-servers: localhost:9092,localhost:9093,localhost:9094
  producer:
    acks: all
    retries: 3
liquibase:
  enabled: true
  change-log: 'classpath:db/changelog/db.changelog-master.xml'

jwt:
  secret: ${JWT_SECRET:vkr-core-transaction-secret-key-must-be-at-least-256-bits-long!}
  expiration-ms: 3600000

auth:
  service-accounts:
    - name: gateway
      roles: [GATEWAY]
      secret: ${GATEWAY_SECRET:gateway-dev-secret-change-me}
    - name: support

```

```

    roles: [SUPPORT]
    secret: ${SUPPORT_SECRET:support-dev-secret-change-me}

outbox:
  partition-count: 8
  poll-interval-ms: 1000
  batch-size: 50
  consumer-group: core-transaction
  lock-ttl-seconds: 30
  cleanup-retention-days: 7
  producer-enabled: true
  consumer-enabled: true

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      application: core-transaction

```

А.2 Точка входа и инфраструктура

```

@SpringBootApplication
@ConfigurationPropertiesScan
@EnableScheduling
class CoreTransactionApplication

fun main(args: Array<String>) {
    runApplication<CoreTransactionApplication>(*args)
}
@Configuration
class KafkaProducerConfig(
    private val kafkaProperties: KafkaProperties,
    private val objectMapper: ObjectMapper
) {

    @Bean
    fun producerFactory(): ProducerFactory<String, Any> {
        val props = mutableMapOf<String, Any>(
            ProducerConfig.BOOTSTRAP_SERVERS_CONFIG to
            kafkaProperties.bootstrapServers,
            ProducerConfig.ACKS_CONFIG to
            (kafkaProperties.producer.acks ?: "all"),
            ProducerConfig.RETRIES_CONFIG to
            (kafkaProperties.producer.retries ?: 3)
        )
    }
}

```

```

        val jsonSerializer = JsonSerializer<Any>(objectMapper)
        return DefaultKafkaProducerFactory(props, StringSerializer(),
jsonSerializer)
    }

    @Bean
    fun kafkaTemplate(producerFactory: ProducerFactory<String, Any>):
KafkaTemplate<String, Any> {
        return KafkaTemplate(producerFactory)
    }
}
@Configuration
class OpenApiConfig(
    @Value("\${server.port:8080}") private val serverPort: Int
) {

    @Bean
    fun openAPI(): OpenAPI = OpenAPI()
        .info(
            Info()
                .title("Core Transaction API")
                .description("Микросервис банковских транзакций –
управление пользователями, счетами и операциями")
                .version("1.0.0")
                .contact(Contact().name("Svyat"))
        )

        .servers(listOf(Server().url("http://localhost:$serverPort").descripti
on("Local"))))
        .components(
            Components()
                .addSecuritySchemes(
                    "bearerAuth",
                    SecurityScheme()
                        .type(SecurityScheme.Type.HTTP)
                        .scheme("bearer")
                        .bearerFormat("JWT")
                )
        )
        .addSecurityItem(SecurityRequirement().addList("bearerAuth"))

    @Bean
    fun authResponsesCustomizer(): GlobalOpenApiCustomizer =
GlobalOpenApiCustomizer { openApi ->
        val errorContent = Content().addMediaType(
            org.springframework.http.MediaType.APPLICATION_JSON_VALUE,
            MediaType().schema(Schema<Any>().`$ref`("#/components/schemas/ErrorRes
ponse"))
        )
        val unauthorized = ApiResponse()

```

```

        .description("Не аутентифицирован – отсутствует или
невалидный JWT")
        .content(errorContent)
        val forbidden = ApiResponse()
        .description("Недостаточно прав (роль не позволяет вызвать
этот endpoint)")
        .content(errorContent)

        openApi.paths?.values?.forEach { pathItem ->
            pathItem.readOperations().forEach { op ->
                val responses = op.responses ?: return@forEach
                if (!responses.containsKey("401"))
responses.addApiResponse("401", unauthorized)
                if (!responses.containsKey("403"))
responses.addApiResponse("403", forbidden)
            }
        }
    }
}

```

A.3 REST API – контроллеры

```

@RestController
class UserController(
    private val userService: UserService
) : UserApi {

    @PreAuthorize("hasRole('GATEWAY')")
    override fun createUser(request: CreateUserRequest):
ResponseEntity<UserResponse> {
        val response = userService.createUser(
            firstName = request.firstName,
            lastName = request.lastName,
            patronymic = request.patronymic,
            phoneNumber = request.phoneNumber,
            email = request.email
        )
        return ResponseEntity(response, HttpStatus.CREATED)
    }

    @PreAuthorize("hasAnyRole('GATEWAY','SUPPORT')")
    override fun getUser(id: Long): ResponseEntity<UserResponse> {
        return ResponseEntity.ok(userService.getUser(id))
    }

    @PreAuthorize("hasAnyRole('GATEWAY','SUPPORT')")
    override fun getUserByPhone(phoneNumber: String):
ResponseEntity<UserResponse> {
        return
ResponseEntity.ok(userService.getUserByPhone(phoneNumber))
    }
}

```

```

@RestController
class TransactionController(
    private val transactionService: TransactionService
) : TransactionApi {

    @PreAuthorize("hasRole('GATEWAY')")
    override fun transferToSavings(request: TransferRequest):
    ResponseEntity<TransactionResponse> {
        return
    ResponseEntity(transactionService.transferToSavings(request),
    HttpStatus.CREATED)
    }

    @PreAuthorize("hasRole('GATEWAY')")
    override fun transferToDeposit(request: TransferRequest):
    ResponseEntity<TransactionResponse> {
        return
    ResponseEntity(transactionService.transferToDeposit(request),
    HttpStatus.CREATED)
    }

    @PreAuthorize("hasRole('GATEWAY')")
    override fun transferToBrokerage(request: TransferRequest):
    ResponseEntity<TransactionResponse> {
        return
    ResponseEntity(transactionService.transferToBrokerage(request),
    HttpStatus.CREATED)
    }

    @PreAuthorize("hasRole('GATEWAY')")
    override fun transferToChecking(request: TransferRequest):
    ResponseEntity<TransactionResponse> {
        return
    ResponseEntity(transactionService.transferToChecking(request),
    HttpStatus.CREATED)
    }

    @PreAuthorize("hasRole('GATEWAY')")
    override fun interbankTransfer(request: InterbankTransferRequest):
    ResponseEntity<TransactionResponse> {
        return
    ResponseEntity(transactionService.interbankTransfer(request),
    HttpStatus.CREATED)
    }

    @PreAuthorize("hasRole('GATEWAY')")
    override fun sbpTransfer(request: SbpTransferRequest):
    ResponseEntity<TransactionResponse> {
        return ResponseEntity(transactionService.sbpTransfer(request),
    HttpStatus.CREATED)
    }
}

```

```

        @PreAuthorize("hasAnyRole('GATEWAY','SUPPORT')")
        override fun processMoneyGift(request: MoneyGiftRequest):
        ResponseEntity<TransactionResponse> {
            return
        ResponseEntity(transactionService.processMoneyGift(request),
        HttpStatus.CREATED)
        }

        @PreAuthorize("hasAnyRole('GATEWAY','SUPPORT')")
        override fun getTransaction(id: Long):
        ResponseEntity<TransactionResponse> {
            return
        ResponseEntity.ok(transactionService.getTransaction(id))
        }

        @PreAuthorize("hasAnyRole('GATEWAY','SUPPORT')")
        override fun getTransactionsByAccount(accountNumber: String):
        ResponseEntity<List<TransactionResponse>> {
            return
        ResponseEntity.ok(transactionService.getTransactionsByAccount(accountN
        umber))
        }
    }
}

```

А.4 Сервисный слой – бизнес-логика

```

@Service
class UserServiceImpl(
    private val userRepository: UserRepository,
    private val userMapper: UserMapper
) : UserService {

    @Transactional
    override fun createUser(
        firstName: String,
        lastName: String,
        patronymic: String?,
        phoneNumber: String,
        email: String?
    ): UserResponse {
        log.info { "Creating user: phone=$phoneNumber" }

        userRepository.findByPhoneNumber(phoneNumber)?.let {
            log.warn { "Duplicate phone number: $phoneNumber" }
            throw BusinessException(HttpStatus.CONFLICT, "User with
        phone number $phoneNumber already exists")
        }
    }
}

```

```

        val user = UserEntity(
            firstName = firstName,
            lastName = lastName,
            patronymic = patronymic,
            phoneNumber = phoneNumber,
            email = email
        )
        val saved = userRepository.save(user)
        log.info { "User created: id=${saved.id}, phone=$phoneNumber"
    }

    return userMapper.toResponse(saved)
}

@Transactional(readOnly = true)
override fun getUser(id: Long): UserResponse {
    log.debug { "Fetching user by id=$id" }
    val user = userRepository.findById(id)
        .orElseThrow { BusinessException(HttpStatus.NOT_FOUND,
            "User with id $id not found") }
    return userMapper.toResponse(user)
}

@Transactional(readOnly = true)
override fun getUserByPhone(phoneNumber: String): UserResponse {
    log.debug { "Fetching user by phone=$phoneNumber" }
    val user = userRepository.findByPhoneNumber(phoneNumber)
        ?: throw BusinessException(HttpStatus.NOT_FOUND, "User
with phone number $phoneNumber not found")
    return userMapper.toResponse(user)
}
}

@Service
class TransactionServiceImpl(
    private val transactionRepository: TransactionRepository,
    private val accountRepository: AccountRepository,
    private val cardRepository: CardRepository,
    private val userRepository: UserRepository,
    private val transactionMapper: TransactionMapper,
    private val meterRegistry: MeterRegistry,
    private val outboxProducer: OutboxProducer
) : TransactionService {

    private fun txCounter(type: TransactionType, status: String):
Counter =
        Counter.builder("transactions.total")
            .tag("type", type.name)
            .tag("status", status)
            .description("Total number of transactions")
            .register(meterRegistry)

```

```

    @Transactional
    override fun transferToSavings(request: TransferRequest):
TransactionResponse {
        return executeInternalTransfer(request,
TransactionType.TRANSFER_SAVINGS, AccountType.SAVINGS)
    }

    @Transactional
    override fun transferToDeposit(request: TransferRequest):
TransactionResponse {
        return executeInternalTransfer(request,
TransactionType.TRANSFER_DEPOSIT, AccountType.DEPOSIT)
    }

    @Transactional
    override fun transferToBrokerage(request: TransferRequest):
TransactionResponse {
        return executeInternalTransfer(request,
TransactionType.TRANSFER_BROKERAGE, AccountType.BROKERAGE)
    }

    @Transactional
    override fun transferToChecking(request: TransferRequest):
TransactionResponse {
        log.info { "Checking-to-checking transfer:
key=${request.idempotencyKey}, src=${request.sourceAccountNumber},
dst=${request.destinationAccountNumber}, amount=${request.amount}" }

        checkIdempotency(request.idempotencyKey)?.let {
            log.info { "Idempotent hit for
key=${request.idempotencyKey}" }
            return it
        }

        if (request.sourceAccountNumber ==
request.destinationAccountNumber) {
            throw BusinessException(
                HttpStatus.BAD_REQUEST,
                "Source and destination accounts must differ"
            )
        }

        val (sourceAccount, destAccount) = lockAccountsInOrder(
            request.sourceAccountNumber,
            request.destinationAccountNumber
        )

        validateBothChecking(sourceAccount, destAccount)
        validateSource(sourceAccount, request.amount,
TransactionType.TRANSFER_CHECKING)
    }

```



```

        return executeDebitCredit(
            sourceAccount, destAccount, request.amount,
            TransactionType.TRANSFER_CHECKING, request.idempotencyKey,
            request.description
        )
    }

    @Transactional
    override fun interbankTransfer(request: InterbankTransferRequest):
    TransactionResponse {
        log.info { "Interbank transfer: key=${request.idempotencyKey},
srcCard=${request.sourceCardNumber},
dstCard=${request.destinationCardNumber}, amount=${request.amount}" }

        checkIdempotency(request.idempotencyKey)?.let {
            log.info { "Idempotent hit for
key=${request.idempotencyKey}" }
            return it
        }

        val sourceCard =
cardRepository.findByCardNumber(request.sourceCardNumber)
            ?: throw BusinessException(HttpStatus.NOT_FOUND, "Source
card ${request.sourceCardNumber} not found")
        val destCard =
cardRepository.findByCardNumber(request.destinationCardNumber)
            ?: throw BusinessException(HttpStatus.NOT_FOUND,
"Destination card ${request.destinationCardNumber} not found")

        if (!sourceCard.isActive) {
            throw BusinessException(HttpStatus.BAD_REQUEST, "Source
card is inactive")
        }
        if (!destCard.isActive) {
            throw BusinessException(HttpStatus.BAD_REQUEST,
"Destination card is inactive")
        }

        val (sourceAccount, destAccount) = lockAccountsInOrder(
            sourceCard.account.accountNumber,
            destCard.account.accountNumber
        )
        validateSource(sourceAccount, request.amount,
TransactionType.INTERBANK_TRANSFER)

        return executeDebitCredit(
            sourceAccount, destAccount, request.amount,
            TransactionType.INTERBANK_TRANSFER,
            request.idempotencyKey, request.description
        )
    }

```

```

    }

    @Transactional
    override fun sbpTransfer(request: SbpTransferRequest):
    TransactionResponse {
        log.info { "SBP transfer: key=${request.idempotencyKey},
srcAccount=${request.sourceAccountNumber},
recipientPhone=${request.recipientPhoneNumber},
amount=${request.amount}" }

        checkIdempotency(request.idempotencyKey)?.let {
            log.info { "Idempotent hit for
key=${request.idempotencyKey}" }
            return it
        }

        val recipientUser =
userRepository.findByPhoneNumber(request.recipientPhoneNumber)
            ?: throw BusinessException(
                HttpStatus.NOT_FOUND,
                "Recipient with phone ${request.recipientPhoneNumber}
not found"
            )

        val destAccountUnlocked =
accountRepository.findByIdAndAccountType(recipientUser.id,
AccountType.CHECKING)
            ?: throw BusinessException(HttpStatus.NOT_FOUND,
"Recipient has no checking account")

        val (sourceAccount, destAccount) = lockAccountsInOrder(
            request.sourceAccountNumber,
            destAccountUnlocked.accountNumber
        )

        if (sourceAccount.user.id == recipientUser.id) {
            throw BusinessException(
                HttpStatus.BAD_REQUEST,
                "SBP transfer to your own account is not allowed"
            )
        }

        if (sourceAccount.accountType != AccountType.CHECKING) {
            throw BusinessException(
                HttpStatus.BAD_REQUEST,
                "SBP transfer requires CHECKING source account, but
was ${sourceAccount.accountType.name}"
            )
        }
    }

```

```

        validateSource(sourceAccount, request.amount,
TransactionType.SBP_TRANSFER)

        return executeDebitCredit(
            sourceAccount, destAccount, request.amount,
            TransactionType.SBP_TRANSFER, request.idempotencyKey,
request.description
        )
    }

    @Transactional
    override fun processMoneyGift(request: MoneyGiftRequest):
TransactionResponse {
        return executeCreditOnly(
            request.idempotencyKey, request.destinationAccountNumber,
request.amount,
            TransactionType.MONEY_GIFT, request.description
        )
    }

    @Transactional(readOnly = true)
    override fun getTransaction(id: Long): TransactionResponse {
        log.debug { "Fetching transaction id=$id" }
        val tx = transactionRepository.findById(id)
            .orElseThrow { BusinessException(HttpStatus.NOT_FOUND,
"Transaction with id $id not found") }
        return transactionMapper.toResponse(tx)
    }

    @Transactional(readOnly = true)
    override fun getTransactionsByAccount(accountNumber: String):
List<TransactionResponse> {
        log.debug { "Fetching transactions for
accountNumber=$accountNumber" }
        return transactionRepository

.findAllBySourceAccountAccountNumberOrDestinationAccountAccountNumber(
accountNumber, accountNumber)
            .map { transactionMapper.toResponse(it) }
    }

    // --- Private helpers ---

    private fun executeInternalTransfer(
        request: TransferRequest,
        type: TransactionType,
        nonCheckingType: AccountType
    ): TransactionResponse {
        log.info { "Internal transfer: key=${request.idempotencyKey},
type=$type, src=${request.sourceAccountNumber},
dst=${request.destinationAccountNumber}, amount=${request.amount}" }
    }

```

```

        checkIdempotency(request.idempotencyKey)?.let {
            log.info { "Idempotent hit for
key=${request.idempotencyKey}" }
            return it
        }

        val (sourceAccount, destAccount) = lockAccountsInOrder(
            request.sourceAccountNumber,
            request.destinationAccountNumber
        )

        validateCheckingPair(sourceAccount, destAccount,
nonCheckingType)
        validateSource(sourceAccount, request.amount, type)

        return executeDebitCredit(
            sourceAccount, destAccount, request.amount, type,
request.idempotencyKey, request.description
        )
    }

    private fun executeCreditOnly(
        idempotencyKey: UUID,
        destinationAccountNumber: String,
        amount: BigDecimal,
        type: TransactionType,
        description: String?
    ): TransactionResponse {
        log.info { "Credit-only: key=$idempotencyKey, type=$type,
dst=$destinationAccountNumber, amount=$amount" }

        checkIdempotency(idempotencyKey)?.let {
            log.info { "Idempotent hit for key=$idempotencyKey" }
            return it
        }

        val destAccount =
accountRepository.findByAccountNumberForUpdate(destinationAccountNumbe
r)

        ?: throw BusinessException(
            HttpStatus.NOT_FOUND,
            "Destination account $destinationAccountNumber not
found"
        )

        if (!destAccount.isActive) {
            throw BusinessException(HttpStatus.BAD_REQUEST,
"Destination account is inactive")
        }
    }

```

```

        destAccount.balance = destAccount.balance.add(amount)
        destAccount.updatedAt = OffsetDateTime.now()
        accountRepository.save(destAccount)

        val transaction = TransactionEntity(
            idempotencyKey = idempotencyKey,
            transactionType = type,
            status = TransactionStatus.COMPLETED,
            destinationAccount = destAccount,
            amount = amount,
            description = description,
            completedAt = OffsetDateTime.now()
        )
        val savedTransaction = transactionRepository.save(transaction)

        txCounter(type, "COMPLETED").increment()

        outboxProducer.publish(
            aggregateType = OutboxAggregateType.TRANSACTION,
            aggregateId = savedTransaction.id.toString(),
            eventType = OutboxEventType.TRANSFER_COMPLETED.name,
            partitionKey = destAccount.accountNumber,
            payload = TransactionOutboxPayload(
                transactionId = savedTransaction.id,
                type = type,
                sourceAccountNumber = null,
                destinationAccountNumber = destAccount.accountNumber,
                amount = amount,
                currency = destAccount.currency,
                status = TransactionStatus.COMPLETED
            )
        )

        log.info { "Transaction completed:
transactionId=${savedTransaction.id}, type=$type, amount=$amount" }

        return transactionMapper.toResponse(savedTransaction)
    }

    private fun executeDebitCredit(
        source: AccountEntity,
        destination: AccountEntity,
        amount: BigDecimal,
        type: TransactionType,
        idempotencyKey: UUID,
        description: String?
    ): TransactionResponse {
        source.balance = source.balance.subtract(amount)
        source.updatedAt = OffsetDateTime.now()
        accountRepository.save(source)
    }

```

```

destination.balance = destination.balance.add(amount)
destination.updatedAt = OffsetDateTime.now()
accountRepository.save(destination)

val transaction = TransactionEntity(
    idempotencyKey = idempotencyKey,
    transactionType = type,
    status = TransactionStatus.COMPLETED,
    sourceAccount = source,
    destinationAccount = destination,
    amount = amount,
    description = description,
    completedAt = OffsetDateTime.now()
)
val saved = transactionRepository.save(transaction)

txCounter(type, "COMPLETED").increment()
meterRegistry.gauge(
    "accounts.balance", listOf(
        Tag.of("accountNumber", source.accountNumber)
    ), source.balance.toDouble()
)

outboxProducer.publish(
    aggregateType = OutboxAggregateType.TRANSACTION,
    aggregateId = saved.id.toString(),
    eventType = OutboxEventType.TRANSFER_COMPLETED.name,
    partitionKey = source.accountNumber,
    payload = TransactionOutboxPayload(
        transactionId = saved.id,
        type = type,
        sourceAccountNumber = source.accountNumber,
        destinationAccountNumber = destination.accountNumber,
        amount = amount,
        currency = source.currency,
        status = TransactionStatus.COMPLETED
    )
)

log.info { "Transaction completed: id=${saved.id}, type=$type,
src=${source.accountNumber}, dst=${destination.accountNumber},
amount=$amount" }

return transactionMapper.toResponse(saved)
}

private fun lockAccountsInOrder(
    accountNumber1: String,
    accountNumber2: String
): Pair<AccountEntity, AccountEntity> {
    val (first, second) = if (accountNumber1 < accountNumber2) {

```

```

        accountNumber1 to accountNumber2
    } else {
        accountNumber2 to accountNumber1
    }

    val firstEntity =
accountRepository.findByAccountNumberForUpdate(first)
        ?: throw BusinessException(HttpStatus.NOT_FOUND, "Account
$first not found")
    val secondEntity =
accountRepository.findByAccountNumberForUpdate(second)
        ?: throw BusinessException(HttpStatus.NOT_FOUND, "Account
$second not found")

    return if (accountNumber1 < accountNumber2) {
        firstEntity to secondEntity
    } else {
        secondEntity to firstEntity
    }
}

private fun validateCheckingPair(
    sourceAccount: AccountEntity,
    destAccount: AccountEntity,
    nonCheckingType: AccountType
) {
    val srcType = sourceAccount.accountType
    val dstType = destAccount.accountType
    val isForward = srcType == AccountType.CHECKING && dstType ==
nonCheckingType
    val isBackward = srcType == nonCheckingType && dstType ==
AccountType.CHECKING
    if (!isForward && !isBackward) {
        throw BusinessException(
            HttpStatus.BAD_REQUEST,
            "Transfer must be between CHECKING and
${nonCheckingType.name} accounts, " +
                "but was ${srcType.name} -> ${dstType.name}"
        )
    }
}

private fun validateBothChecking(
    sourceAccount: AccountEntity,
    destAccount: AccountEntity
) {
    val srcType = sourceAccount.accountType
    val dstType = destAccount.accountType
    if (srcType != AccountType.CHECKING || dstType !=
AccountType.CHECKING) {
        throw BusinessException(

```

```

        HttpStatus.BAD_REQUEST,
        "Both accounts must be CHECKING, but was
        ${srcType.name} -> ${dstType.name}"
    )
}

private fun validateSource(account: AccountEntity, amount:
BigDecimal, type: TransactionType) {
    if (!account.isActive) {
        log.warn { "Source account ${account.accountNumber} is
inactive" }
        throw BusinessException(HttpStatus.BAD_REQUEST, "Source
account is inactive")
    }
    if (account.balance < amount) {
        log.warn { "Insufficient funds:
accountNumber=${account.accountNumber}, available=${account.balance},
requested=$amount" }
        txCounter(type, "FAILED").increment()
        throw BusinessException(
            HttpStatus.BAD_REQUEST,
            "Insufficient funds: available ${account.balance},
requested $amount"
        )
    }
}

private fun checkIdempotency(key: UUID): TransactionResponse? {
    return transactionRepository.findByIdempotencyKey(key)?.let {
        transactionMapper.toResponse(it)
    }
}

//back/src/main/java/ru/diplom/backend/service/AttachmentService.java
@Service
@SuppressWarnings("null")
public class AttachmentService {

    private static final Logger log =
LoggerFactory.getLogger(AttachmentService.class);

    private final AttachmentRepository attachmentRepository;
    private final MessageRepository messageRepository;
    private final Path uploadBasePath;

    public AttachmentService(AttachmentRepository
attachmentRepository,
                           MessageRepository messageRepository,

```



```

        @Value("${app.uploads.dir:uploads}")
String uploadsDir) {
    this.attachmentRepository = attachmentRepository;
    this.messageRepository = messageRepository;
    this.uploadBasePath = Path.of(uploadsDir).toAbsolutePath();
    try {
        Files.createDirectories(uploadBasePath);
    } catch (IOException e) {
        throw new RuntimeException("Не удалось создать каталог
загрузок: " + uploadBasePath, e);
    }
}

    public Attachment upload(UUID messageId, UUID ownerId,
    MultipartFile file) throws IOException {
        Message m = messageRepository.findById(messageId)
            .orElseThrow(() -> new
    IllegalArgumentException("Письмо не найдено"));
        if (!m.getOwnerId().equals(ownerId)) {
            throw new IllegalArgumentException("Письмо не найдено");
        }
        String originalName =
    Objects.requireNonNullElse(file.getOriginalFilename(), "file");
        String safeName = UUID.randomUUID() + "_" +
    originalName.replaceAll("[^a-zA-Z0-9._-]", "_");
        Path dir = uploadBasePath.resolve(messageId.toString());
        Files.createDirectories(dir);
        Path target = dir.resolve(safeName);
        file.transferTo(target.toFile());
        String relativePath = messageId + "/" + safeName;
        return attachmentRepository.insert(messageId, originalName,
    relativePath,
            file.getContentType(), file.getSize());
    }

    public List<Attachment> listByMessage(UUID messageId) {
        return attachmentRepository.findByMessageId(messageId);
    }

    public Attachment getById(UUID attachmentId) {
        return attachmentRepository.findById(attachmentId)
            .orElseThrow(() -> new
    IllegalArgumentException("Вложение не найдено"));
    }

    public Path getFilePath(Attachment a) {
        return uploadBasePath.resolve(a.getFilePath());
    }

    public boolean isOwner(Attachment attachment, UUID ownerId) {

```

```

        Message m =
messageRepository.findById(attachment.getMessageId()).orElse(null);
        return m != null && m.getOwnerId().equals(ownerId);
    }

    public void deleteById(UUID messageId) {
        List<Attachment> attachments =
attachmentRepository.findById(messageId);
        for (Attachment a : attachments) {
            Path filePath = getFilePath(a);
            try {
                Files.deleteIfExists(filePath);
            } catch (IOException e) {
                log.warn("Не удалось удалить файл вложения: {}",
filePath, e);
            }
        }
        Path messageDir =
uploadBasePath.resolve(messageId.toString());
        try {
            Files.deleteIfExists(messageDir);
        } catch (IOException e) {
            log.warn("Не удалось удалить директорию вложений: {}",
messageDir, e);
        }
        attachmentRepository.deleteById(messageId);
    }
}

//back/src/main/java/ru/diplom/backend/service/AuthService.java
@Service
public class AuthService {

    private static final String[] SYSTEM_FOLDER_CODES = {"INBOX",
"SENT", "DRAFTS", "TRASH", "SPAM"};
    private static final String[] SYSTEM_FOLDER_NAMES = {"Входящие",
"Отправленные", "Черновики", "Корзина", "Спам"};

    private final UserRepository userRepository;
    private final UserSettingsRepository userSettingsRepository;
    private final FolderRepository folderRepository;
    private final PasswordEncoder passwordEncoder = new
BCryptPasswordEncoder();

    public AuthService(UserRepository userRepository,
UserSettingsRepository userSettingsRepository,
FolderRepository folderRepository) {
        this.userRepository = userRepository;
        this.userSettingsRepository = userSettingsRepository;
        this.folderRepository = folderRepository;
    }
}

```

```

    @Transactional
    public User register(String email, String password, String
displayName) {
        if (userRepository.existsByEmail(email)) {
            throw new IllegalArgumentException("Пользователь с таким
email уже зарегистрирован");
        }
        String hash = passwordEncoder.encode(password);
        User user = userRepository.insert(email, hash, displayName);
        userSettingsRepository.insert(user.getId(), "light", "");
        for (int i = 0; i < SYSTEM_FOLDER_CODES.length; i++) {
            folderRepository.insert(user.getId(),
SYSTEM_FOLDER_NAMES[i], SYSTEM_FOLDER_CODES[i]);
        }
        user.setPasswordHash(null);
        return user;
    }

    public User login(String email, String password) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new
IllegalArgumentException("Неверный email или пароль"));
        if (!passwordEncoder.matches(password,
user.getPasswordHash())) {
            throw new IllegalArgumentException("Неверный email или
пароль");
        }
        user.setPasswordHash(null);
        return user;
    }

    public User findById(UUID userId) {
        return userRepository.findById(userId)
            .orElseThrow(() -> new
IllegalArgumentException("Пользователь не найден"));
    }

    public void changePassword(UUID userId, String currentPassword,
String newPassword) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new
IllegalArgumentException("Пользователь не найден"));
        if (!passwordEncoder.matches(currentPassword,
user.getPasswordHash())) {
            throw new IllegalArgumentException("Неверный текущий
пароль");
        }
        String newHash = passwordEncoder.encode(newPassword);
        userRepository.updatePasswordHash(userId, newHash);
    }

```

```
}
```

A.5 Репозитории – доступ к БД

```
interface OutboxConsumerOffsetRepository :  
JpaRepository<OutboxConsumerOffsetEntity, Long> {  
  
    fun findByConsumerGroupAndPartitionNum(consumerGroup: String,  
partitionNum: Int): OutboxConsumerOffsetEntity?  
  
    @Transactional  
    @Modifying  
    @Query(  
        ""  
        INSERT INTO outbox_consumer_offsets (consumer_group,  
partition_num, last_offset, updated_at)  
VALUES (:group, :partition, :offset, NOW())  
ON CONFLICT (consumer_group, partition_num)  
DO UPDATE SET last_offset = :offset, updated_at = NOW()  
        ""  
        ,  
        nativeQuery = true  
    )  
    fun upsertOffset(  
        @Param("group") consumerGroup: String,  
        @Param("partition") partitionNum: Int,  
        @Param("offset") lastOffset: Long  
    )  
}
```

```
interface OutboxPartitionLockRepository :  
JpaRepository<OutboxPartitionLockEntity, Long> {  
  
    @Transactional  
    @Modifying  
    @Query(  
        ""  
        INSERT INTO outbox_partition_locks (consumer_group,  
partition_num, locked_by, locked_at, expires_at)  
VALUES (:group, :partition, :lockedBy, NOW(), :expiresAt)  
ON CONFLICT (consumer_group, partition_num)  
DO UPDATE SET locked_by = :lockedBy, locked_at = NOW(),  
expires_at = :expiresAt  
WHERE outbox_partition_locks.expires_at < NOW()  
OR outbox_partition_locks.locked_by = :lockedBy  
        ""  
        ,  
        nativeQuery = true  
    )  
    fun tryAcquireLock(  

```

```

        @Param("group") consumerGroup: String,
        @Param("partition") partitionNum: Int,
        @Param("lockedBy") lockedBy: String,
        @Param("expiresAt") expiresAt: OffsetDateTime
    ): Int

    @Transactional
    @Modifying
    @Query(
        """
        DELETE FROM outbox_partition_locks
        WHERE consumer_group = :group
          AND partition_num = :partition
          AND locked_by = :lockedBy
        """,
        nativeQuery = true
    )
    fun releaseLock(
        @Param("group") consumerGroup: String,
        @Param("partition") partitionNum: Int,
        @Param("lockedBy") lockedBy: String
    )

    @Transactional
    @Modifying
    @Query("DELETE FROM outbox_partition_locks WHERE expires_at < NOW()", nativeQuery = true)
    fun deleteExpiredLocks(): Int
}
signature != null ? signature : "");

```

A.6 Kafka – доставка

```

@Configuration
class KafkaProducerConfig(
    private val kafkaProperties: KafkaProperties,
    private val objectMapper: ObjectMapper
) {

    @Bean
    fun producerFactory(): ProducerFactory<String, Any> {
        val props = mutableMapOf<String, Any>(
            ProducerConfig.BOOTSTRAP_SERVERS_CONFIG to
            kafkaProperties.bootstrapServers,
            ProducerConfig.ACKS_CONFIG to
            (kafkaProperties.producer.acks ?: "all"),
            ProducerConfig.RETRIES_CONFIG to
            (kafkaProperties.producer.retries ?: 3)
        )
        val jsonSerializer = JsonSerializer<Any>(objectMapper)
    }
}

```

```

        return DefaultKafkaProducerFactory(props, StringSerializer(),
jsonSerializer)
    }

    @Bean
    fun kafkaTemplate(producerFactory: ProducerFactory<String, Any>):
KafkaTemplate<String, Any> {
        return KafkaTemplate(producerFactory)
    }
}

@Component
class TransactionEventHandler(
    private val kafkaTemplate: KafkaTemplate<String, Any>,
    private val objectMapper: ObjectMapper
) : OutboxEventHandler {

    override fun supportedAggregateTypes() =
listOf(OutboxAggregateType.TRANSACTION)

    override fun handle(eventType: String, payload: String, metadata:
OutboxEventMetadata) {
        val transactionPayload = objectMapper.readValue(payload,
TransactionOutboxPayload::class.java)
        val idempotencyKey = "${metadata.aggregateId}-
${metadata.messageId}"

        val message = TransactionKafkaMessage(
            idempotencyKey = idempotencyKey,
            eventType = eventType,
            payload = transactionPayload,
            metadata = KafkaEventMetadata(
                aggregateId = metadata.aggregateId,
                partitionKey = metadata.partitionKey,
                sourceMessageId = metadata.messageId,
                createdAt = metadata.createdAt
            )
        )

        val result = kafkaTemplate.send(TOPIC, metadata.aggregateId,
message)
            .get(SEND_TIMEOUT_SECONDS, TimeUnit.SECONDS)

        log.info {
            "Sent to Kafka: topic=$TOPIC, key=${metadata.aggregateId},
" +
                "offset=${result.recordMetadata.offset()},
idempotencyKey=$idempotencyKey"
        }
    }
}

```

```

    companion object {
        const val TOPIC = "core.transactions.request"
        private const val SEND_TIMEOUT_SECONDS = 10L
    }
}

@Component
class TransactionEventConsumer(
    private val transactionLedgerService: TransactionLedgerService,
    private val meterRegistry: MeterRegistry
) {

    @KafkaListener(
        topics =
["\\${kafka.consumer.topic:core.transactions.request}"],
        groupId = "\\${kafka.consumer.group-id:core-ledger-group}",
        containerFactory = "kafkaListenerContainerFactory"
    )
    fun consume(
        record: ConsumerRecord<String, TransactionKafkaMessage>,
        acknowledgment: Acknowledgment
    ) {
        val message = record.value()
        log.info { "Received event: idempotencyKey=${message.idempotencyKey}, type=${message.eventType}" }

        val saved = transactionLedgerService.processEvent(message)
        if (saved) {
            meterRegistry.counter(
                "ledger.events.processed",
                "event_type", message.eventType,
                "status", message.payload.status.name
            ).increment()
        } else {
            meterRegistry.counter("ledger.events.duplicates").increment()
            log.debug { "Duplicate event skipped: idempotencyKey=${message.idempotencyKey}" }
        }

        acknowledgment.acknowledge()
    }
}

```

A.7 Модели

```

@Entity
@Table(name = "accounts")

```

```

class AccountEntity(

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    val user: UserEntity,

    @Column(name = "account_number", nullable = false, unique = true,
length = 20)
    val accountNumber: String,

    @Enumerated(EnumType.STRING)
    @Column(name = "account_type", nullable = false, length = 30)
    val accountType: AccountType,

    @Column(name = "currency", nullable = false, length = 3)
    val currency: String = "RUB",

    @Column(name = "balance", nullable = false, precision = 19, scale =
4)
    var balance: BigDecimal = BigDecimal.ZERO,

    @Column(name = "is_active", nullable = false)
    var isActive: Boolean = true,

    @Column(name = "created_at", nullable = false, updatable = false)
    val createdAt: OffsetDateTime = OffsetDateTime.now(),

    @Column(name = "updated_at", nullable = false)
    var updatedAt: OffsetDateTime = OffsetDateTime.now(),

    @OneToMany(mappedBy = "account", fetch = FetchType.LAZY)
    val cards: MutableList<CardEntity> = mutableListOf()
)

@Entity
@Table(name = "outbox_messages")
class OutboxMessageEntity(

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,

    @Column(name = "aggregate_type", nullable = false, length = 100)
    val aggregateType: String,

    @Column(name = "aggregate_id", nullable = false, length = 100)
    val aggregateId: String,

```



```

@Column(name = "event_type", nullable = false, length = 100)
val eventType: String,

@Column(name = "partition_key", nullable = false, length = 100)
val partitionKey: String,

@Column(name = "partition_num", nullable = false)
val partitionNum: Int,

@JdbcTypeCode(SqlTypes.JSON)
@Column(name = "payload", nullable = false, columnDefinition =
"jsonb")
val payload: String,

@Column(name = "created_at", nullable = false, updatable = false)
val createdAt: OffsetDateTime = OffsetDateTime.now()
)

@Entity
@Table(name = "transactions")
class TransactionEntity(

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long = 0,

    @Column(name = "idempotency_key", nullable = false, unique = true)
    val idempotencyKey: UUID,

    @Enumerated(EnumType.STRING)
    @Column(name = "transaction_type", nullable = false, length = 30)
    val transactionType: TransactionType,

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    var status: TransactionStatus = TransactionStatus.PENDING,

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "source_account_id")
    val sourceAccount: AccountEntity? = null,

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "destination_account_id")
    val destinationAccount: AccountEntity? = null,

    @Column(name = "amount", nullable = false, precision = 19, scale =
4)
    val amount: BigDecimal,

    @Column(name = "currency", nullable = false, length = 3)

```

```

    val currency: String = "RUB",

    @Column(name = "description", length = 500)
    val description: String? = null,

    @Column(name = "error_message", length = 500)
    var errorMessage: String? = null,

    @Column(name = "created_at", nullable = false, updatable = false)
    val createdAt: OffsetDateTime = OffsetDateTime.now(),

    @Column(name = "completed_at")
    var completedAt: OffsetDateTime? = null
)

```

A.8 DTO – объект передачи данных

```

data class TransactionKafkaMessage(
    val idempotencyKey: String,
    val eventType: String,
    val payload: TransactionOutboxPayload,
    val metadata: KafkaEventMetadata
)

data class KafkaEventMetadata(
    val aggregateId: String,
    val partitionKey: String,
    val sourceMessageId: Long,
    val createdAt: OffsetDateTime
)

data class TransactionOutboxPayload(
    val transactionId: Long,
    val type: TransactionType,
    val sourceAccountNumber: String?,
    val destinationAccountNumber: String,
    val amount: BigDecimal,
    val currency: String,
    val status: TransactionStatus
)

data class TransferRequest(
    @field:NotNull
    val idempotencyKey: UUID,

    @field:NotBlank
    val sourceAccountNumber: String,

```

```

    @field:NotBlank
    val destinationAccountNumber: String,

    @field:NotNull
    @field:DecimalMin(value = "0.01")
    val amount: BigDecimal,

    val description: String? = null
)

```

A.9 Модульные тесты

```

class TransactionEventHandlerIntegrationTest : IntegrationTestBase() {

    @Autowired
    private lateinit var transactionEventHandler:
    TransactionEventHandler

    companion object {
        private val messageIdCounter = AtomicLong(System.nanoTime())

        @JvmStatic
        val kafka: KafkaContainer =
        KafkaContainer("apache/kafka:3.7.0")
            .apply { start() }

        @JvmStatic
        @DynamicPropertySource
        fun kafkaProperties(registry: DynamicPropertyRegistry) {
            registry.add("spring.kafka.bootstrap-servers") {
                kafka.bootstrapServers }
        }

        private fun uniqueMessageId(): Long =
        messageIdCounter.incrementAndGet()

        private fun createStringConsumer(): KafkaConsumer<String, String> {
            val props = mapOf(
                ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG to
                kafka.bootstrapServers,
                ConsumerConfig.GROUP_ID_CONFIG to "test-consumer-
                ${System.nanoTime()}",
                ConsumerConfig.AUTO_OFFSET_RESET_CONFIG to "earliest",
                ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG to
                StringDeserializer::class.java.name,
                ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG to
                StringDeserializer::class.java.name
            )
            return KafkaConsumer(props)
        }
    }
}

```

```

private fun pollAllMessages(
    consumer: KafkaConsumer<String, String>,
    timeoutMs: Long = 10_000
): List<ConsumerRecord<String, String>> {
    val results = mutableListOf<ConsumerRecord<String, String>>()
    val deadline = System.currentTimeMillis() + timeoutMs
    while (System.currentTimeMillis() < deadline) {
        val records = consumer.poll(Duration.ofSeconds(1))
        records.forEach { results.add(it) }
        if (results.isNotEmpty() && records.isEmpty) break
    }
    return results
}

private fun parseMessage(json: String): TransactionKafkaMessage =
    objectMapper.readValue(json,
TransactionKafkaMessage::class.java)

private fun samplePayload() = TransactionOutboxPayload(
    transactionId = 42,
    type = TransactionType.TRANSFER_SAVINGS,
    sourceAccountNumber = "10000000000000000001",
    destinationAccountNumber = "20000000000000000001",
    amount = BigDecimal("1500.00"),
    currency = "RUB",
    status = TransactionStatus.COMPLETED
)

private fun sampleMetadata(
    messageId: Long = uniqueMessageId(),
    aggregateId: String = "42"
) = OutboxEventMetadata(
    messageId = messageId,
    aggregateType = "TRANSACTION",
    aggregateId = aggregateId,
    partitionKey = "10000000000000000001",
    partitionNum = 0,
    createdAt = OffsetDateTime.now()
)

@Nested
inner class SendToKafka {

    @Test
    fun `handler sends message to kafka topic with correct structure`() {
        val msgId = uniqueMessageId()
        val payload = samplePayload()
        val metadata = sampleMetadata(messageId = msgId)
        val payloadJson = objectMapper.writeValueAsString(payload)
    }
}

```

```

        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, metadata)

        val consumer = createStringConsumer()
        consumer.subscribe(listOf(TransactionEventHandler.TOPIC))
        try {
            val records = pollAllMessages(consumer)
            val message = records
                .map { parseMessage(it.value()) }
                .first { it.idempotencyKey == "42-$msgId" }

            assertEquals("TRANSFER_COMPLETED", message.eventType)
            assertEquals(42L, message.payload.transactionId)
            assertEquals(TransactionType.TRANSFER_SAVINGS,
message.payload.type)
            assertEquals(BigDecimal("1500.00"),
message.payload.amount)
            assertEquals("RUB", message.payload.currency)
            assertEquals(TransactionStatus.COMPLETED,
message.payload.status)
            assertEquals("10000000000000000001",
message.payload.sourceAccountNumber)
            assertEquals("20000000000000000001",
message.payload.destinationAccountNumber)

            assertEquals("42", message.metadata.aggregateId)
            assertEquals("10000000000000000001",
message.metadata.partitionKey)
            assertEquals(msgId, message.metadata.sourceMessageId)

            val record = records.first {
                parseMessage(it.value()).idempotencyKey == "42-$msgId" }
            assertEquals("42", record.key())
        } finally {
            consumer.close()
        }
    }

    @Test
    fun `handler sends multiple messages preserving order`() {
        val msgId1 = uniqueMessageId()
        val msgId2 = uniqueMessageId()
        val msgId3 = uniqueMessageId()
        val payload = samplePayload()
        val payloadJson = objectMapper.writeValueAsString(payload)

        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId1))
        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId2))
    }

```

```

        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId3))

        val consumer = createStringConsumer()
        consumer.subscribe(listOf(TransactionEventHandler.TOPIC))
        try {
            val records = pollAllMessages(consumer)
            val expectedKeys = setOf("42-$msgId1", "42-$msgId2",
"42-$msgId3")
            val messages = records
                .map { parseMessage(it.value()) }
                .filter { it.idempotencyKey in expectedKeys }

            assertEquals(3, messages.size, "Expected 3 messages")
            assertEquals("42-$msgId1", messages[0].idempotencyKey)
            assertEquals("42-$msgId2", messages[1].idempotencyKey)
            assertEquals("42-$msgId3", messages[2].idempotencyKey)
        } finally {
            consumer.close()
        }
    }
}

@Nested
inner class IdempotencyKey {

    @Test
    fun `same outbox message produces deterministic idempotency
key`() {
        val msgId = uniqueMessageId()
        val payload = samplePayload()
        val payloadJson = objectMapper.writeValueAsString(payload)
        val metadata = sampleMetadata(messageId = msgId)

        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, metadata)
        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, metadata)

        val consumer = createStringConsumer()
        consumer.subscribe(listOf(TransactionEventHandler.TOPIC))
        try {
            val records = pollAllMessages(consumer)
            val messages = records
                .map { parseMessage(it.value()) }
                .filter { it.idempotencyKey == "42-$msgId" }

            assertEquals(2, messages.size, "Expected 2 messages with
same idempotency key")
            assertEquals(messages[0].idempotencyKey,
messages[1].idempotencyKey)

```

```

        } finally {
            consumer.close()
        }
    }

    @Test
    fun `different outbox messages produce different idempotency
keys`() {
        val msgId1 = uniqueMessageId()
        val msgId2 = uniqueMessageId()
        val payload = samplePayload()
        val payloadJson = objectMapper.writeValueAsString(payload)

        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId1))
        transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId2))

        val consumer = createStringConsumer()
        consumer.subscribe(listOf(TransactionEventHandler.TOPIC))
        try {
            val records = pollAllMessages(consumer)
            val expectedKeys = setOf("42-$msgId1", "42-$msgId2")
            val messages = records
                .map { parseMessage(it.value()) }
                .filter { it.idempotencyKey in expectedKeys }

            assertEquals(2, messages.size)
            assertNotEquals(messages[0].idempotencyKey,
messages[1].idempotencyKey)
        } finally {
            consumer.close()
        }
    }

    @Nested
    inner class PayloadDeserialization {

        @Test
        fun `handler correctly deserializes outbox payload string to
object`() {
            val msgId = uniqueMessageId()
            val payloadJson = """
            {
                "transactionId": 99,
                "type": "SBP_TRANSFER",
                "sourceAccountNumber": "10000000000000000005",
                "destinationAccountNumber": "20000000000000000010",
                "amount": 250.50,
                "currency": "RUB",
            """

```

```

        "status": "PENDING"
    }
    """.trimIndent()

    transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId))

    val consumer = createStringConsumer()
    consumer.subscribe(listOf(TransactionEventHandler.TOPIC))
    try {
        val records = pollAllMessages(consumer)
        val message = records
            .map { parseMessage(it.value()) }
            .first { it.idempotencyKey == "42-$msgId" }

        assertEquals(99L, message.payload.transactionId)
        assertEquals(TransactionType.SBP_TRANSFER,
message.payload.type)
        assertEquals("10000000000000000005",
message.payload.sourceAccountNumber)
        assertEquals("20000000000000000010",
message.payload.destinationAccountNumber)
        assertEquals(BigDecimal("250.50"),
message.payload.amount)
        assertEquals("RUB", message.payload.currency)
        assertEquals(TransactionStatus.PENDING,
message.payload.status)
    } finally {
        consumer.close()
    }
}

@Test
fun `handler correctly deserializes credit-only payload with
null source`() {
    val msgId = uniqueMessageId()
    val payloadJson = """
    {
        "transactionId": 100,
        "type": "MONEY_GIFT",
        "sourceAccountNumber": null,
        "destinationAccountNumber": "20000000000000000001",
        "amount": 1000.00,
        "currency": "RUB",
        "status": "COMPLETED"
    }
    """.trimIndent()

    transactionEventHandler.handle("TRANSFER_COMPLETED",
payloadJson, sampleMetadata(messageId = msgId))

```



```

        val consumer = createStringConsumer()
        consumer.subscribe(listOf(TransactionEventHandler.TOPIC))
        try {
            val records = pollAllMessages(consumer)
            val message = records
                .map { parseMessage(it.value()) }
                .first { it.idempotencyKey == "42-$msgId" }

            assertNull(message.payload.sourceAccountNumber)
            assertEquals(TransactionType.MONEY_GIFT,
message.payload.type)
        } finally {
            consumer.close()
        }
    }
}

@Nested
inner class ErrorHandling {

    @Test
    fun `handler throws exception on invalid payload json`() {
        val invalidJson = "not a json"
        val metadata = sampleMetadata()

        assertThrows(Exception::class.java) {
            transactionEventHandler.handle("TRANSFER_COMPLETED",
invalidJson, metadata)
        }
    }
}

class TransactionIntegrationTest : IntegrationTestBase() {

    @Autowired
    private lateinit var api: TestApiClient

    private lateinit var checkingAccountNumber: String
    private lateinit var savingsAccountNumber: String
    private var checkingCardNumber: String? = null
    private var userId: Long = 0

    @BeforeEach
    fun setUp() {
        api.setAuthToken(authToken)
        val accounts = api.createUserWithCheckingAndSavings()
        userId = accounts.userId
        checkingAccountNumber = accounts.checkingAccountNumber
        savingsAccountNumber = accounts.savingsAccountNumber
        checkingCardNumber = accounts.checkingCardNumber
    }
}

```

```

        api.fundAccount(checkingAccountNumber)
    }

    @Nested
    inner class TransferToSavings {

        @Test
        fun `success and balances updated`() {
            val request = TestDataFactory.transferRequest(
                checkingAccountNumber, savingsAccountNumber,
                amount = BigDecimal("3000.00"), description = "На
накопления"
            )

            authPost("/api/v1/transactions/savings", request).andExpect
            {
                status { isCreated() }
                jsonPath("$.transactionType") {
value("TRANSFER_SAVINGS") }
                jsonPath("$.status") { value("COMPLETED") }
                jsonPath("$.amount") { value(3000.0) }
            }

            authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
                jsonPath("$.balance") { value(7000.0) }
            }

            authGet("/api/v1/accounts/$savingsAccountNumber").andExpect {
                jsonPath("$.balance") { value(3000.0) }
            }
        }

        @Test
        fun `idempotency returns same response`() {
            val request = TestDataFactory.transferRequest(
                checkingAccountNumber, savingsAccountNumber, amount =
BigDecimal("1000.00")
            )

            authPost("/api/v1/transactions/savings", request).andExpect
            { status { isCreated() } }

            authPost("/api/v1/transactions/savings", request).andExpect
            {
                status { isCreated() }
                jsonPath("$.transactionType") {
value("TRANSFER_SAVINGS") }
            }
        }
    }
}

```

```

authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
    jsonPath("$.balance") { value(9000.0) }
}

@Test
fun `insufficient funds returns 400`() {
    val request = TestDataFactory.transferRequest(
        checkingAccountNumber, savingsAccountNumber, amount =
BigDecimal("999999.00")
    )

    authPost("/api/v1/transactions/savings", request).andExpect
{
    status { isBadRequest() }
    jsonPath("$.message") { exists() }
}

@Test
fun `both checking accounts returns 400`() {
    val request = TestDataFactory.transferRequest(
        checkingAccountNumber, checkingAccountNumber, amount =
BigDecimal("100.00")
    )

    authPost("/api/v1/transactions/savings", request).andExpect
{
    status { isBadRequest() }
}

@Test
fun `backward direction (savings to checking) success`() {
    authPost("/api/v1/transactions/savings",
TestDataFactory.transferRequest(
    checkingAccountNumber, savingsAccountNumber, amount =
BigDecimal("4000.00")
)).andExpect { status { isCreated() } }

    val backward = TestDataFactory.transferRequest(
        savingsAccountNumber, checkingAccountNumber,
        amount = BigDecimal("1500.00"), description = "Возврат
на расчётный"
    )

    authPost("/api/v1/transactions/savings",
backward).andExpect {
        status { isCreated() }
    }
}

```

```

        jsonPath("$.transactionType") {
value("TRANSFER_SAVINGS") }
        jsonPath("$.sourceAccountNumber") {
value(savingsAccountNumber) }
        jsonPath("$.destinationAccountNumber") {
value(checkingAccountNumber) }
    }

    authGet("/api/v1/accounts/$savingsAccountNumber").andExpect {
        jsonPath("$.balance") { value(2500.0) }
    }

    authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
        jsonPath("$.balance") { value(7500.0) }
    }
}

@Nested
inner class TransferToDeposit {

    @Test
    fun `success and balances updated`() {
        val depositAccountNumber = api.createAccount(userId,
"DEPOSIT")

        val request = TestDataFactory.transferRequest(
            checkingAccountNumber, depositAccountNumber,
            amount = BigDecimal("2000.00"), description = "На вклад"
        )

        authPost("/api/v1/transactions/deposit", request).andExpect
{
            status { isCreated() }
            jsonPath("$.transactionType") {
value("TRANSFER_DEPOSIT") }
            jsonPath("$.status") { value("COMPLETED") }
        }

        authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
            jsonPath("$.balance") { value(8000.0) }
        }

        authGet("/api/v1/accounts/$depositAccountNumber").andExpect {
            jsonPath("$.balance") { value(2000.0) }
        }
    }
}

```

```

@Nested
inner class TransferToBrokerage {

    @Test
    fun `success and balances updated`() {
        val brokerageAccountNumber = api.createAccount(userId,
"BROKERAGE")

        val request = TestDataFactory.transferRequest(
            checkingAccountNumber, brokerageAccountNumber,
            amount = BigDecimal("5000.00"), description = "Ha
брокерский счёт"
        )

        authPost("/api/v1/transactions/brokerage",
request).andExpect {
            status { isCreated() }
            jsonPath("$.transactionType") {
value("TRANSFER_BROKERAGE") }
            jsonPath("$.status") { value("COMPLETED") }
        }

        authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
            jsonPath("$.balance") { value(5000.0) }
        }

        authGet("/api/v1/accounts/$brokerageAccountNumber").andExpect {
            jsonPath("$.balance") { value(5000.0) }
        }
    }
}

@Nested
inner class TransferChecking {

    @Test
    fun `success and balances updated`() {
        val secondCheckingNumber = api.createAccount(userId,
"CHECKING")

        val request = TestDataFactory.transferRequest(
            checkingAccountNumber, secondCheckingNumber,
            amount = BigDecimal("2500.00"), description = "Между
расчётными"
        )

        authPost("/api/v1/transactions/checking",
request).andExpect {

```

```

        status { isCreated() }
        jsonPath("$.transactionType") {
value("TRANSFER_CHECKING") }
        jsonPath("$.status") { value("COMPLETED") }
        jsonPath("$.amount") { value(2500.0) }
    }

    authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
        jsonPath("$.balance") { value(7500.0) }
    }

    authGet("/api/v1/accounts/$secondCheckingNumber").andExpect {
        jsonPath("$.balance") { value(2500.0) }
    }
}

@Test
fun `same source and destination returns 400`() {
    val request = TestDataFactory.transferRequest(
        checkingAccountNumber, checkingAccountNumber, amount =
BigDecimal("100.00")
    )

    authPost("/api/v1/transactions/checking",
request).andExpect {
        status { isBadRequest() }
    }
}

@Test
fun `non-checking destination returns 400`() {
    val request = TestDataFactory.transferRequest(
        checkingAccountNumber, savingsAccountNumber, amount =
BigDecimal("100.00")
    )

    authPost("/api/v1/transactions/checking",
request).andExpect {
        status { isBadRequest() }
    }
}

@Nested
inner class MoneyGift {

    @Test
    fun `credits destination account`() {
        val request = TestDataFactory.moneyGiftRequest(

```

```

        savingsAccountNumber, amount = BigDecimal("5000.00"),
description = "Подарок"
    )

    authPost("/api/v1/transactions/gift", request).andExpect {
        status { isCreated() }
        jsonPath("$.transactionType") { value("MONEY_GIFT") }
        jsonPath("$.sourceAccountNumber") { doesNotExist() }
        jsonPath("$.destinationAccountNumber") {
value(savingsAccountNumber) }
    }

    authGet("/api/v1/accounts/$savingsAccountNumber").andExpect {
        jsonPath("$.balance") { value(5000.0) }
    }
}

@Nested
inner class SbpTransfer {

    @Test
    fun `success and balances updated`() {
        val recipientUserId = api.createUser(phoneNumber =
"+79997654321")
        val recipientAccountNumber =
api.createAccount(recipientUserId, "CHECKING")

        val request = TestDataFactory.sbpTransferRequest(
checkingAccountNumber, "+79997654321", amount =
BigDecimal("2000.00"), description = "Перевод по СБП"
        )

        authPost("/api/v1/transactions/sbp", request).andExpect {
            status { isCreated() }
            jsonPath("$.transactionType") { value("SBP_TRANSFER") }
            jsonPath("$.status") { value("COMPLETED") }
            jsonPath("$.amount") { value(2000.0) }
        }

        authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
            jsonPath("$.balance") { value(8000.0) }
        }

        authGet("/api/v1/accounts/$recipientAccountNumber").andExpect {
            jsonPath("$.balance") { value(2000.0) }
        }
    }
}

```

```

@Test
fun `recipient not found returns 404`() {
    val request = TestDataFactory.sbpTransferRequest(
        checkingAccountNumber, "+700000000000"
    )

    authPost("/api/v1/transactions/sbp", request).andExpect {
        status { isNotFound() }
    }
}

@Test
fun `insufficient funds returns 400`() {
    val recipientUserId = api.createUser(phoneNumber =
"+79997654321")
    api.createAccount(recipientUserId, "CHECKING")

    val request = TestDataFactory.sbpTransferRequest(
        checkingAccountNumber, "+79997654321", amount =
BigDecimal("999999.00")
    )

    authPost("/api/v1/transactions/sbp", request).andExpect {
        status { isBadRequest() }
    }
}

@Test
fun `recipient phone belongs to sender returns 400 and balance
unchanged`() {
    val request = TestDataFactory.sbpTransferRequest(
        checkingAccountNumber, "+79991234567", amount =
BigDecimal("2000.00")
    )

    authPost("/api/v1/transactions/sbp", request).andExpect {
        status { isBadRequest() }
    }

    authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
        jsonPath("$.balance") { value(10000.0) }
    }
}

@Nested
inner class InterbankTransfer {

    @Test

```



```

        fun `success and balances updated`() {
            val sourceCardNumber = checkingCardNumber!!

            val recipientUserId = api.createUser(phoneNumber =
"+79997654321")
            val recipient = api.createAccountWithCard(recipientUserId,
"CHECKING")
            val recipientCardNumber = recipient.cardNumber!!

            val request = TestDataFactory.interbankTransferRequest(
                sourceCardNumber, recipientCardNumber,
                amount = BigDecimal("1500.00"), description =
"Межбанковский перевод"
            )

            authPost("/api/v1/transactions/interbank",
request).andExpect {
                status { isCreated() }
                jsonPath("$.transactionType") {
value("INTERBANK_TRANSFER") }
                jsonPath("$.status") { value("COMPLETED") }
                jsonPath("$.amount") { value(1500.0) }
            }

            authGet("/api/v1/accounts/$checkingAccountNumber").andExpect {
                jsonPath("$.balance") { value(8500.0) }
            }

            authGet("/api/v1/accounts/${recipient.accountNumber}").andExpect {
                jsonPath("$.balance") { value(1500.0) }
            }
        }

        @Test
        fun `source card not found returns 404`() {
            val request = TestDataFactory.interbankTransferRequest(
                "9999999999999999", checkingCardNumber!!
            )

            authPost("/api/v1/transactions/interbank",
request).andExpect {
                status { isNotFound() }
            }
        }

        @Test
        fun `insufficient funds returns 400`() {
            val recipientUserId = api.createUser(phoneNumber =
"+79997654321")

```

```

        val recipient = api.createAccountWithCard(recipientUserId,
"CHECKING")

        val request = TestDataFactory.interbankTransferRequest(
            checkingCardNumber!!, recipient.cardNumber!!, amount =
BigDecimal("999999.00")
        )

        authPost("/api/v1/transactions/interbank",
request).andExpect {
            status { isBadRequest() }
        }
    }

    @Nested
    inner class GetTransaction {

        @Test
        fun `success`() {
            val request = TestDataFactory.moneyGiftRequest(
                checkingAccountNumber, amount = BigDecimal("100.00"),
description = "test"
            )

            val result = authPost("/api/v1/transactions/gift",
request).andReturn()

            val txId =
objectMapper.readTree(result.response.contentAsString)["id"].asLong()

            authGet("/api/v1/transactions/$txId").andExpect {
                status { isOk() }
                jsonPath("$.id") { value(txId) }
                jsonPath("$.status") { value("COMPLETED") }
            }
        }

        @Test
        fun `not found returns 404`() {
            authGet("/api/v1/transactions/999999").andExpect {
                status { isNotFound() }
            }
        }

        @Test
        fun `by account - returns related transactions`() {
            repeat(3) {
                api.fundAccount(checkingAccountNumber,
BigDecimal("100.00"))
            }
        }
    }
}

```

```
authGet("/api/v1/transactions/account/$checkingAccountNumber").andExpect {  
    status { isOk() }  
    jsonPath("$.length()") { value(4) }  
}
```